POLITECNICO
MILANO 1863

Requirements Analysis and Specification Document

(RASD)

Riccardo Cerberi
10775421

Matteo Gatti
10840092

Alessio Ginolfi
10797951

A.Y. 2024/2025 - Software engineering 2

Contents

1 Introduction **3**
- 1.1 Purpose . . . 3
  - 1.1.1 Goals . . . 3
- 1.2 Scope . . . 3
  - 1.2.1 World phenomena . . . 3
  - 1.2.2 Shared phenomena . . . 4
- 1.3 Definitions, Acronyms, Abbreviations . . . 5
  - 1.3.1 Interviewer . . . 5
  - 1.3.2 Candidate . . . 5
  - 1.3.3 Project . . . 5
  - 1.3.4 Curriculum Vitae . . . 6
  - 1.3.5 Tag . . . 6
  - 1.3.6 Statistical Analysis Tool . . . 6
  - 1.3.7 Suggestion Process . . . 6
  - 1.3.8 Video Conference Tool . . . 6
  - 1.3.9 Recommendation Process . . . 6
  - 1.3.10 Matching Process . . . 7
  - 1.3.11 Selection Process . . . 7
  - 1.3.12 Acronyms . . . 8
- 1.4 Revision history . . . 8
- 1.5 Reference documents . . . 9
- 1.6 Document structure . . . 9

2 Overall description **10**
- 2.1 Product perspective . . . 10
  - 2.1.1 Scenarios . . . 10
  - 2.1.2 Class diagram . . . 12
  - 2.1.3 State diagrams . . . 14
- 2.2 Product functions . . . 15
- 2.3 User characteristics . . . 17
  - 2.3.1 Student . . . 18
  - 2.3.2 Company Member . . . 18
  - 2.3.3 University Member . . . 18
- 2.4 Assumptions, dependencies and constraints . . . 18

3 Specific Requirements **20**
- 3.1 External Interface Requirements . . . 20
  - 3.1.1 User Interfaces . . . 20
  - 3.1.2 Hardware Interfaces . . . 20
  - 3.1.3 Software Interfaces . . . 20
  - 3.1.4 Communication Interfaces . . . 20
- 3.2 Functional Requirements . . . 20
- 3.3 Use cases . . . 22
  - 3.3.1 Use case diagrams . . . 22
  - 3.3.2 Use cases in details . . . 27
- 3.4 Mapping on requirements . . . 71
- 3.5 Performance Requirements . . . 72
  - 3.5.1 Response time . . . 72

3.5.2 Scalability . . . 72
3.5.3 Data volume . . . 73
3.5.4 Latency . . . 73
3.6 Design Constraints . . . 73
3.6.1 Privacy Complaints . . . 73
3.6.2 Hardware limitations . . . 73
3.7 Software System Attributes . . . 73
3.7.1 Reliability . . . 73
3.7.2 Availability . . . 73
3.7.3 Security . . . 73
3.7.4 Maintainability . . . 74
3.7.5 Portability . . . 74

4 Formal analysis using Alloy **75**
4.1 Pred Show1 Run Example . . . 82

5 Effort spent **85**

6 References **86**

1 Introduction

1.1 Purpose

The goal of the **Students&Companies** system is to provide a platform where students and companies can manage and monitor all processes related to internships. Internships are proposed by companies, and students can submit an application by accessing the system. Additionally, companies can also initiate the process by sending a match request to a student. Students and companies find each other through a recommendation system. The platform provides advice to students regarding the curriculum vitae and to companies regarding the internship descriptions. There is also a third actor involved: the university. The university is responsible for tracking the status of the internships, only for its enrolled students. If necessary, the university can terminate an internship if it identifies significant issues.

1.1.1 Goals

This section outlines the goals of the system.

- [G1] The company members aims to spread the project proposals to students.
- [G2] The student would like to find proposals that are most suited to him.[1]
- [G3] The company members aim to identify student CVs that align with the requirements of their internship proposals.[2]
- [G4] The student and the company members would like to receive suggestions on how to improve CVs and project descriptions.
- [G5] The university members would like to monitor the status of internships involving all their students.[3]
- [G6] The company members and the students aim to carry out the matching and selection processes.
- [G7] The company members and the student engaged in an ongoing internship wish to keep track of its status.

1.2 Scope

In this section, the phenomena are presented: world phenomena (WP) and shared phenomena (SPM stands for machine controlled and SPW stands for world controlled). There are no machine phenomena (MP).

1.2.1 World phenomena

- [WP1] The student writes his CV.
- [WP2] The company member considers a new project to propose to students and therefore it defines an accurate and clear description including also terms and requirements for the internship.

[1] This will be carried out by recommendation system.
[2] This will be carried out by recommendation system.
[3] Universities members may interrupt internships if it's needed.

• [WP3] The student works on the tasks assigned by the company.

• [WP4] The company member makes an interview to the student.[4]

• [WP5] The external statistical analysis tool calculates statistics between tags, in order to find recommendations.

• [WP6] The company member prepares a structured questionnaire that will be filled out by the interviewers.[5]

• [WP7] The university member handles the complaints of enrolled students.

• [WP8] The company member reviews the data collected with structured questionnaires from the open selection processes.

• [WP9] The company member or the student does not adhere to the policies established in the internship contract, causing an issue for the counterpart.

1.2.2 Shared phenomena

Machine controlled

• [SPM1] The student receives a notify, through an email, about the availability of an internship that might interest him.

• [SPM2] The company members receive a notify, through an email, about the availability of a new CV that aligns with their interests.

• [SPM3] The company members receive a notify, through an email, about a new application for an internship.

• [SPM4] The system generates a link to the virtual room using an external video conferencing tool.

World controlled

• [SPW1] The user registers into the system by providing all required information.

• [SPW2] The student publishes his CV.

• [SPW3] The student look at available internship offers.

• [SPW4] The company member publishes a new internship proposal.

• [SPW5] The company member communicates to the candidate the date and time of the interview, along with other necessary information (how it has to be done, rules, duration).[6]

• [SPW6] The company member selects students which are the best fit for the proposed internship.

• [SPW7] After concluding internship, the student or the company member publishes feedback regarding the counterpart.[7]

[4] Interviews may be either remotely (by external tools e.g. Webex) or in person.

[5] The structured questionnaire is managed externally from the platform for the entire duration of the selection process.

[6] Counterparts will communicate through an integrated chat

[7] Users can submit feedback only after the internship they participated in has ended.

• [SPW8] The company member or the student submits a complaint or reports an issue regarding the ongoing internship.

• [SPW9] The university member monitors the status of an ongoing student's internship by looking on reports and complaints.

• [SPW10] The university member interrupts a student's internship.

• [SPW11] The student confirm his institutional email by accessing his mailbox.

• [SPW12] The company member confirm his company email by accessing his mailbox.

• [SPW13] The university member confirm his university email by accessing his mailbox.

• [SPW14] The university member creates his university by providing all the information.

• [SPW15] The company member creates his company by providing all the information.

• [SPW16] The company member or the student posts a new report regarding the ongoing internship.

• [SPW17] The company member or the student looks on new reports posted by counterpart.

• [SPW18] The student submits a matching proposal to company members for an internship.

• [SPW19] The company member sends its internship proposal to a student.

• [SPW20] The external statistical analysis tool sends to the system the recommendations related to the user of interest (student or company member).

1.3 Definitions, Acronyms, Abbreviations

1.3.1 Interviewer

An interviewer is a company member responsible for leading interviews, either online or in person. Their user account is identical to that of other company members, as their role does not impact the system's functionality.

1.3.2 Candidate

A candidate for a specific internship refers to a student who has successfully matched with a company for that internship. Their user account remains a standard student account and is not treated as a different user type.

1.3.3 Project

A project is a term used to refer to an internship offered by a company.

1.3.4 Curriculum Vitae

The Curriculum Vitae of students is a fundamental part of their profile. It is composed of a file uploaded by the student and a structured version represented by tags. This serves as the main source of information for companies to evaluate candidates.

1.3.5 Tag

A tag is a term that describes a particular skill or role possessed by a student or required for an internship. Tags are used in the structured version of the students' curriculum and also appear in internship offers to enhance the clarity of the internship's topic and improve the accuracy of recommendations. Examples of tags include "Software Engineer", "Data Scientist", or "User Interface Designer". Each pair of tags starts with an initial compatibility score, which is later updated for specific companies as the system receives feedback. The primary purpose of tags is to supply the statistical analysis tool with structured information, along with feedback, enabling it to update compatibility scores. In conclusion, tags serve as a concise and standardized source of information, enhancing the system's ability to refine matching and evaluation processes.

1.3.6 Statistical Analysis Tool

The Statistical Analysis Tool is an external application used to calculate compatibility between students and internships. Initially, the system provides the tool with a compatibility score for each pair of tags. The tool is aware of all internships and students, along with their corresponding sets of tags. After receiving feedback, the tool updates the compatibility scores for the tags involved in the feedback, but only for the specific counterparts associated with it, without affecting the global compatibility of the tags. Additionally, the tool is responsible for providing personalized recommendations to students and company members at random intervals.

1.3.7 Suggestion Process

The suggestion process provides simple suggestions to both companies and students. Suggestions contain standard tips to improve internship offers or the students' structured curriculum and are mainly focused on adding tags or brief descriptions related to the provided tags.

1.3.8 Video Conference Tool

The Video Conference Tool is an external application used during online interviews. The system interacts with it by creating a virtual meeting room and generating a link to that room, where the interviewer and the candidate, who have been arranged for the online interview, can communicate (ex: WebEx, Microsoft Teams).

1.3.9 Recommendation Process

The recommendation process is composed by two main functionality:

- **HomePage recommendation :** This function is responsible for receiving and displaying the recommendations generated by the Statistical Analysis Tool. These recommendations are presented to the user (either a company or a student) on their main page immediately after logging in, without requiring any

additional action. The suggestions can include internship proposals for students and CVs that meet the requirements of previously published projects for companies. There is also a notification process associated with this functionality, that sends an email to the user whenever a new recommendation becomes available.

- **Search :** This function can only be performed by a student and provides the results of a search after entering a keyword. The results consist of a list of available internship proposals, determined through keyword matching and statistical analysis produced by an external tool. If any internships that contain the keyword are also included in the recommended list, they are sorted based on the compatibility score calculated by the external Statistical Analysis Tool; otherwise, the results are displayed in alphabetical order.

1.3.10 Matching Process

This is the process responsible for matching students with companies.

- **Student point of view :** The student selects an internship, identified through the "Homepage Recommendation" or "Search," and submits an application to the respective company. The student may also receive a request from a company that considers their curriculum eligible for a specific internship it offers. In this case, the student has the option to either accept or decline the proposal.
- **Company member point of view :** The company receives an application from a student and can either accept or decline it. The company may also invite students with an eligible curriculum, identified through the "Homepage Recommendation", to participate in an internship it offers.

Both the student and the company members can send and receive multiple requests over the same period of time. Each match proposal references the related internship offer, the sender, and the receiver. If the students initiate the request, the receiver will be the company member who published the internship offer. On the other hand, if the request is initiated by the company member, the student will be the receiver. When the receiver of a request evaluates it, the matching process is complete. If the receiver responds positively, the selection process begins.

1.3.11 Selection Process

The purpose of this process is to provide support throughout the entire selection phase, which concludes when an eligible group of students, whose number is equal to the available places, is chosen from among all the candidates. Students who are not selected will receive a notification of their unsuccessful application. Once a match is finalized, the system creates a chat channel to facilitate communication between the company member and the selected student. Through this channel, the two parties can arrange an interview, which may take place either online or in person. In both cases, the software automatically generates a reminder when the interview is scheduled. This reminder contains different information depending on the type of interview agreed upon.

- **In-person interview:** The system provides a feature in order to add the location and other details to the reminder of the scheduled interview.
- **Online interview:** The system generates the videoconference link and appends it to the reminder of the scheduled interview when the agreed time arrives. The

link is created using an external videoconferencing tool, which will also be used to conduct the interview.

After the interview concludes, the system provides a feature to select the best candidates among the interviewed students. The selection will be based on the company's decision, but the process will be managed solely by the company member who published the internship offer to which the selection refers. Once the company makes its choice, all other candidates receive a notification informing them of the negative outcome.

1.3.12 Acronyms

Below there are the acronyms used in the document along with their respective definitions.

- **S&C**: The Students&Companies system. This is the name of the platform to be developed.
- **G**: Goal.
- **WP**: World phenomenon
- **SPM**: Shared phenomenon - machine controlled.
- **SPW**: Shared phenomenon - world controlled.
- **DA**: Domain assumption.
- **R**: Requirement.
- **UC**: Use case.

1.4 Revision history

Below there are listed all the document versions in chronological order, starting from the most recent to the oldest.

- **Version 2.0** - 06/01/2025
  - "Definitions, Acronyms, Abbreviations" section: enhanced the explanation of the matchmaking and selection processes.
  - UC3: it was previously in conflict with the domain assumption DA11. Now, in UC3, it is assumed that the university of the student is already registered in the system.
  - UC14: the request has been removed from the server. Suggestions are already displayed by the system, and the student applies the modifications directly.
  - UC15: the request has been removed from the server. Suggestions are already displayed by the system, and the company member applies the modifications directly.
  - UC16: removed retrieve students info and minor changes.
  - UC17: removed retrieve company member info and minor changes.
  - UC18: minor changes.

  - UC19: minor changes.
  - UC16,...,UC28: make the functions synchronous.
  - UC25: replace ST as CM in the sequence diagram.
  - UC26: add writecomplaint() function to start the interaction with the system.
  - UC28: change name of the function signature from the system to the external statistical analysis tool.

- **Version 1.0** - 22/12/2024

1.5 Reference documents

This document has been written based on the project description provided in the assignment. It also conforms to IEEE standards for writing requirements and specifications documents.

1.6 Document structure

This section presents the structure of the document.

1. **Introduction**: this section introduces the system's domain through the definition of the phenomena. Additionally, it was decided to explain the functioning of certain parts of the system in order to make the document more clear.
2. **Overall description**: this section describes the domain in more detail, presenting scenarios, class diagrams, state diagrams, product functions, assumptions and user characteristics.
3. **Specific requirements**: this section lists the requirements (both functional and non-functional) and the use cases.
4. **Formal analysis using Alloy**: this section shows the modeling of a part of the system created with Alloy, along with the corresponding results obtained from the analyzer.
5. **Effort spent**: this section presents the work hours spent by each team member.
6. **References**: this section mentions all the external documents or tools used for the development of the project.

2 Overall description

2.1 Product perspective

2.1.1 Scenarios

- **Company announces a new internship**

  Code S.P.A announces a new software engineering internship in Italy. The company secretary logs into the Student & Company platform and drafts a brief description of the intern's tasks in the designated text box, adding a list of role requirements. The system flags the absence of a language requirement with the prompt: *"No language requirement?"*. Acting on this suggestion, the secretary includes proficiency in English as a standard requirement. Once finalized, the secretary clicks the *"Open Internship"* button, making the position available for student applications.

- **Marco subscribes to the platform**

  Marco creates an account on the platform by entering his personal information, including his institutional email and a password. He clicks the *"Create your CV"* button to prepare his CV, following the on-screen guidelines. As a master's student in computer science, Marco adds to his profile the software engineering tag.

- **Sending a match proposal**

  Marco logs into the platform, eager to secure an internship. On the homepage, he notices a notification: *"Available software engineering internship at Code S.P.A"*. Interested, Marco clicks the *"Send Request"* button, submitting his application to Code S.P.A.

- **Match accepted**

  The next day, Anna logs into the platform and sees a notification: *"New Candidate"* regarding Marco's application. Curious, she clicks the *"Show Request"* button to review the details. Impressed by his qualifications, she selects *"Add Candidate"* to shortlist him. The system automatically generates a private chat, and Anna sends him a message: *"Hi, I am Anna, recruiter at Code S.P.A. Can we schedule an interview?"*.

- **Chatting**

  Marco receives Anna's message and uses the platform's messaging feature to confirm the interview date and format. They agree on an online interview, which Anna schedules by selecting the *"Online"* option. Both receive a notification: *"Meeting scheduled for 11 a.m. on Wednesday"*. To ensure everything is correctly arranged, Anna checks the reminder, which shows the meeting details along with the virtual room link.

- **New interns are notified**

  After conducting all interviews, the recruiter updates the system with the final decisions. Marco is notified that he has been accepted for the internship. Thrilled, Marco logs in and clicks the *"Confirm"* button.

- **Student keeps track of the internship**

Marco begins his internship with enthusiasm, contributing to significant company projects. After successfully submitting his first pull request, he documents his achievement in a report.

- **Student writes a complaint**

  Marco is not the only student enrolled in the internship program at Code S.P.A; Pino is also working there in a different department. While Marco enjoys the experience, Pino is struggling because his assigned tasks do not align with the job description, not even remotely. Frustrated, Pino uses the *"Write Complaint"* button to document the issue and sends the complaint to his university via the *"Send Complaint"* button.

- **University monitors ongoing internships**

  Zeno, the university's internship coordinator, reviews Pino's complaint and investigates the issue. Upon noticing similar complaints from other students, Zeno contacts Daniele, the company's internship manager, to address the concerns. Daniele assures Zeno that the company will strive to align assigned tasks more closely with the job descriptions.

- **Interruption of an internship**

  Despite Pino's complaint, nothing changes. He continues to receive unrelated tasks, which seem even more arbitrary, as though his supervisor is retaliating. Unable to bear the situation any longer, Pino writes a detailed two-page complaint describing his experience. Zeno receives the complaint and decides to terminate Pino's internship by selecting the *"Interrupt Internship"* button. Pino is notified of the termination and decides to provide feedback to ensure no other student endures a similar experience.

- **End of the internship**

  The internship concludes. Marco logs into the platform and notices a notification about the internship's completion. He clicks the *"Yes"* button on the notification to provide feedback. Marco writes a detailed review, highlighting the rewarding experience of being entrusted with significant projects and responsibilities. Finally, he submits his feedback by clicking the *"Post"* button.

2.1.2 Class diagram

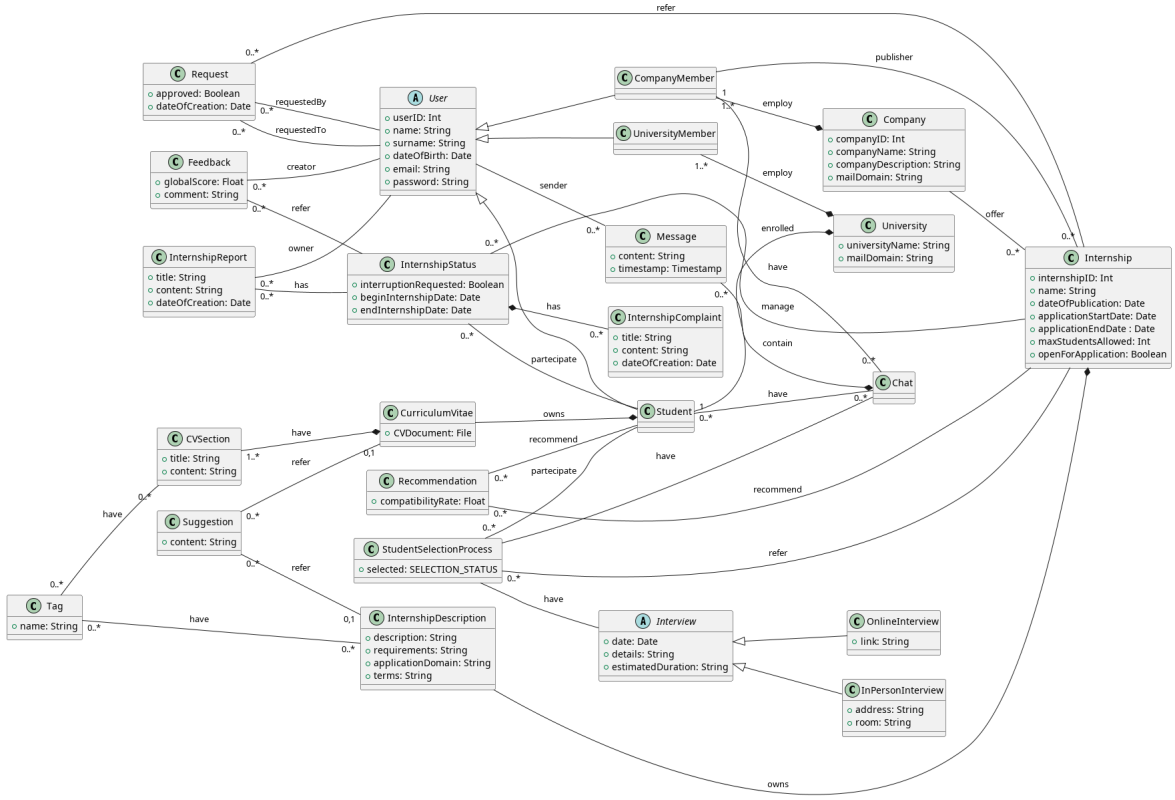

Figure 1: Domain class diagram.

The previous diagram aims to model all relevant aspects of the system mentioned so far (e.g., in the phenomena). Almost all the cardinalities of type "1" have been omitted to improve the readability of the diagram.

The first key concept is the user. **User** class represents a generic and abstract physical actor interacting with the system. A user account (email and password) is associated with it, enabling authentication in the system. There are three types of users: **Student**, **CompanyMember**, and **UniversityMember**. Each "member" is then associated with a specific institution (**University** or **Company**, depending on the user type).

The **Company** and **University** classes describe various institutions. The field "mailDomain" specifies the email domain associated with the company or university (e.g., "polimi.it"). This field is used to validate the registration of new members associated with specific company or new students associated with university. The system will compare the domain of the email entered by the new user with the one specified by the organization. If the comparison is successful, the user will be accepted; otherwise, the registration will be canceled.

The **CurriculumVitae** class represents a student's curriculum. The field "CV-Document" allows students to upload and store their curriculum in the system as a file (e.g., in pdf format). The type "File" represents the abstract concept of a digital document. This file can be accessed by company members during the selection process. Companies can also access a structured version of the same curriculum. A structured version refers to one compiled by the student directly within the system, allowing the definition of various sections. The **CVSection** class models this concept. It represents a specific experience undertaken by the student (either academic/university-related or work-related), such as language certificates, diplomas, and publications. This structured curriculum version has been introduced (in addition to the uploaded file) to perform the recommendation system. Using tags (specified by the **Tag** class) added by the student, the system will calculate affinity indicators between students and internships using an external tool. Internships also include Tags, which in this case are added by company members. To conclude, recommendation will consider curriculum and internships tags and feedbacks posted by students and company members.

The **Internship** class represents an internship proposal created by a company. A few clarifications about its attributes: "maxStudentsAllowed" is the number of students the company aims to recruit and "openForApplication" specifies whether applications are currently allowed. Each internship proposal can involve multiple students. For this reason, the **InternshipStatus** class is introduced. It represents an instance of an internship for a specific student. This class contains all the information regarding the internship's progress, ensuring that each participant (student, company, and university) stays informed about its status.

There are three classes associated to the InternshipStatus class: InternshipComplaint, Feedback, and InternshipReport.
InternshipComplaint models a complaint created by a student and directed to the company. **Feedback** represents the final evaluation written by the student and/or the company about the other party at the end of the internship. The "globalScore" field stores the overall score. These evaluations are used by the system to generate recommendations, through the external statistical analysis tool. **InternshipReport** represents a report written by a student or the company. A report is a brief description of what occurred during the internship on a specific day.

The **Chat** class describes communication between the student and the company member. It is instantiated only after the match is confirmed by both the two parties.

The **StudentSelectionProcess** class handles all stages of selecting an **individual** student, from matching to communicating the results (including interviews). The outcome referred to a specific student is represented by the attribute "selected" that can take the following values: *NOT_DEFINED*, *YES* or *NO*. As long as the company members do not select candidates for the internship, the "selected" field will be *NOT_DEFINED*.

Additionally, the **Request** class models a "match request" made by either the student or the company and directed to the other party. If both parties accept, the attribute "approved" will be set to true and then the selection process begins.

Finally, the **Suggestion** class represents tips that the system provides, either to

students (e.g., for their CV) or to companies (e.g., for internship descriptions).

2.1.3 State diagrams

This section presents the state diagrams corresponding to selected classes from the previously introduced class diagram. We thought to model the diagrams associated with the Internship, InternshipStatus and StudentSelectionProcess classes to better clarify their lifecycle within the system. Additionally, these classes represent the core concepts of the system.

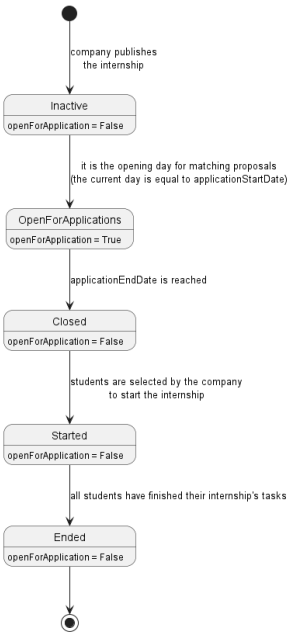

Figure 2: State diagram of Internship class.

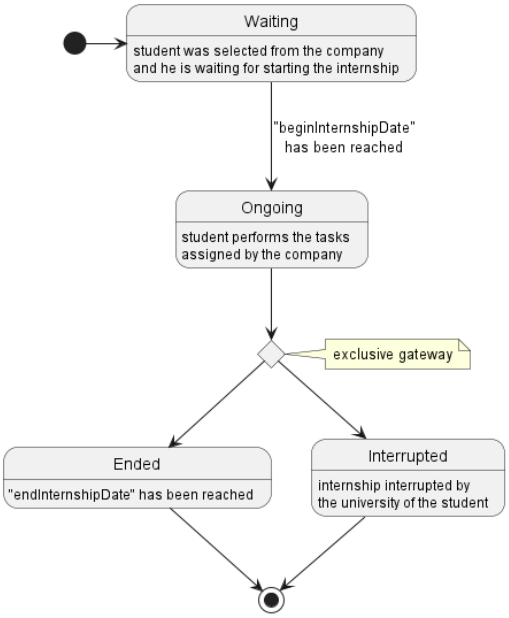

Figure 3: State diagram of InternshipStatus class.

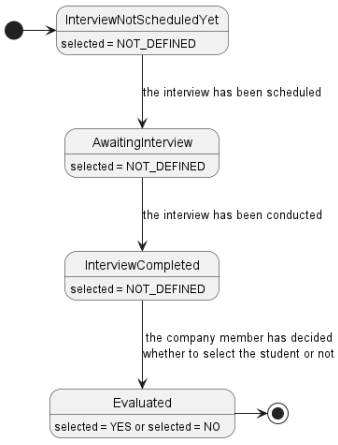

Figure 4: State diagram of StudentSelectionProcess.

2.2 Product functions

The product functions are presented. They represent the main features that the system must support.

- **Register to the system**
  A user who wishes to use the platform must register by providing their first name, last name, an email address, date of birth and a password that will be used for authentication in subsequent logins. If the user is a student or a university member, they must provide their institutional email address. On the other hand, if the user is a company member, they must specify a company domain email address. Additionally, students and university members will be required to specify their university, while company members must indicate the company they work for. Once all the inserted information is confirmed, the system will send an email to the specified address to verify ownership of the email account. The user must check their inbox and confirm the address. In any case, the system will compare the domain of the provided email address with the official domain of the institution. If they do not match, the registration will be denied. This product function assumes that the institution (company or university) associated with the user already exists within the system.

- **Login to the system**
  User submits his credentials (email and password) and enter the system. Credentials must be correct in order to access the homepage of the system.

- **Add a company into the system**
  A company decides to use the system. A member accesses the platform and registers by providing their personal details (same as described in the "Register to the system" product function). Once the required information is entered, the system will prompt the user to create the associated company, as no email domain is linked to the provided email address. The user will then input the company's name followed by a brief description (the email domain associated with the company will be automatically set by the system). At this point, the company will appear in the system along with the company member, who will immediately be able to publish internships.

- **Add a university into the system**
  A university decides to use the system. A member accesses the platform and registers by providing their personal details (same as described in the "Register to the system" product function). Once the required information is entered, the system will prompt the user to create the associated university, as no email domain is linked to the provided email address. The user will then input the university's name (the email domain associated with the university will be automatically set by the system). Once the information entered is confirmed, the university will be added to the system, allowing its students to register on the platform and search for internships.

- **Notify recommendations**
  Periodically, the system notifies students and company members of the various recommendations identified by the external statistical analysis tool.

The following product functions require the users to be authenticated in the system before they are executed.

- **Create an internship**
  The company member publishes a new internship to be shared with students by providing: a name, the valid application period, the maximum number of students allowed, the description, the requirements, the domain of application, tags and the terms specified in the contract (e.g., type of contract and compensation). Thus, after that, students will be able to view the new proposal submitted by the company (by searching through keywords or thanks to the recommendation system).

- **Upload a new version of the curriculum**
  The student accesses their personal area and uploads the updated version of their document (e.g., in PDF format) to the system, so that companies can view the updated CV.

- **Edit the digitalized version of the curriculum** The student accesses their personal area to edit or add a section to their digital curriculum. Once all changes have been made, the student posts the new version of the digital CV.

- **Edit internship information**
  The company member logs into the system to edit information related to an internship previously created. They navigate to the page containing the list of internships published by the company and select the one to be edited. After making all the necessary changes, the user saves the updated version of the internship.

- **Searching for an internship**
  The student logs into the platform to browse the available internships. To do this, they enter keywords to filter the results and view them. Alternatively, the student can rely on the recommendation system, which will provide a list of internships aligned with their interests, with no need to specify keywords anymore.

- **Send matching requests to the counterpart**
  The student (or company member) sends a match request to the counterpart after viewing the internship proposal (the student's curriculum). Once the request is made, the user waits for a response from the counterpart.

- **Send a message to the counterpart**
  The student (or company member) sends a message to the counterpart. To do this, they log into the platform, select the corresponding chat channel, write their message, and finally send it.

- **Schedule interviews**
  The student and the company member have agreed on the day, time, and type of the interview by using chat. The company member then logs into the platform and sets the date and time for the interview. If the interview is of type "online", the system will automatically generate a link to access the virtual room using an external tool. The link will be displayed to both parties in the proper section.

- **Show the status of a selection process**
  The company member wants to consult the information related to the selection

process for an internship. To do this, they access the platform and go to the page of the internship. At this point, the system will display a list of candidates and information regarding the interviews. The student will only be able to access information related to their own selection process. For example, they will be able to check the outcome of the selection and understand whether they have been accepted or not.

- **Finalize the selection process** The company member enters the platform and selects the most suitable students for the proposed internship by consulting the list of all the candidates and the related information. Once the students are selected, all candidates will be notified about the outcome of the selection.

- **Manage reports**
  The student (or the company member) publishes a report through a proper section of the platform. In the report they describe the current status of the internship and the events that occurred throughout the day. This part of the system ensures that all parties involved are kept informed about the progress of the internship. When a report is uploaded to the platform by the student, it will be made available to the company's members and the student's university members.

- **Manage feedbacks**
  The student (or the company member) submits feedback through the dedicated section of the platform. The feedback will include an overall evaluation of the experience. This feedback will not be shared with the counterpart or the university members but is intended to enhance the recommendation process. Providing feedback is optional but strongly encouraged by the system.

- **Manage complaints**
  The student wants to write a complaint regarding the company that offered the internship, so they log into the platform. They navigate to the section related to the internship and enter the description of the complaint. After confirming, the complaint will be added to the system. The university member will then be able to view the new complaint in the specific section.

- **Provide suggestions**
  After the student has completed the digital version of their curriculum, the system will provide them with a series of suggestions on how to improve and expand their CV, so that it can be more easily found by companies relevant to them. One suggestion could be to add missing information (for example tags) in a particular section. The suggestions are also provided to company members. However, these will focus on the information related to the internship provided by company members.

- **Interrupt an internship**
  The university member accesses the system and, after reviewing the complaints from a student regarding a particular internship, decides to terminate the activity. Once the decision is confirmed on the platform, both parties involved (the student and the company member) will be notified.

2.3 User characteristics

There are three types of users: Students, Company Members, and University Members. Each user is identified by an email and a password, and once registered, they

can log in using only these two credentials.

2.3.1 Student

The student registers by providing their name, surname, an email associated with the university they belong to (registered on the platform), and a password. The student joins the Students&Companies platform to find an internship that fulfills their needs. They can apply to multiple internship proposals, and if accepted, they can track their experiences through their account on the platform. Each student account is linked to the same university associated with their email and is responsible for handling their complaints.

2.3.2 Company Member

The company member is a user who represents an employee of a company registered on the Students&Companies platform. They register by providing their name, surname, an email from their company's domain, and a password. A company member can publish internships offered by their company, review student applications, communicate with applicants to arrange interviews. After the interviews are concluded, company members will select the candidates who will be hired for the internship.

2.3.3 University Member

The university member is a user who represents a worker at a university registered on the Students&Companies platform. A university member handles all student complaints and can interrupt an internship when necessary.

2.4 Assumptions, dependencies and constraints

This section outlines the domain assumptions.

- [DA1] The statistical analysis tool produces reliable analysis.
- [DA2] The student writes only correct information inside the CV.
- [DA3] The university member correctly interrupts internships, only if it is strictly necessary.
- [DA4] The video conference tool correctly creates the room for interviewer and student.
- [DA5] The company member properly writes the description (including terms and requirements) of the internship.
- [DA6] The interviewer correctly conducts the interview.
- [DA7] The company member selects students following proper criteria: company doesn't discriminate candidates based on their age, gender, ethnicity and selects the most deserving candidates.
- [DA8] The company member schedules the interview based on the time and date agreed with the student.
- [DA9] An interviewer from the company attend to the scheduled interview.

• [DA10] During the registration process, the user provides only correct information.

• [DA11] When a student registers, the associated university must already exist in the system (previously created).

• [DA12] University member who registers his university into the system inserts correct information.

• [DA13] Company member who registers his company into the system inserts correct information.

3 Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The Students&Companies platform has a web application interface accessible to all students, company members, and university members via an internet browser.

3.1.2 Hardware Interfaces

Each user can access the platform from any device equipped with an internet browser and a reliable internet connection. The most common devices used to access the system are smartphones, tablets, and computers.

3.1.3 Software Interfaces

The system requires the APIs of two external tools: the video conferencing tool and the statistical analysis tool. Additionally, it will use a mailing system to verify users and send updates about new recommendations.

3.1.4 Communication Interfaces

The system's required communication interfaces include the HTTPS protocol and the Simple Mail Transfer Protocol (SMTP) for email handling.

3.2 Functional Requirements

Below there are the functional requirements that the system must guarantee.

- [R1] The system must permit students to register by providing their personal information and an institutional email.[8]
- [R2] The system must permit company members to register.
- [R3] The system must permit universities members to register.
- [R4] The system must permit universities members to register their universities.
- [R5] The system must permit companies members to register their companies.
- [R6] The system must allow users to log in by inserting their credentials.
- [R7] The system must allow student to publish their CV.
- [R8] The system must allow companies members to publish internships.
- [R9] The system shall inform students when suitable internships are available through recommendation.
- [R10] The system shall inform companies members about the availability of student CVs corresponding to their needs, through recommendation.
- [R11] The system must use analysis produced by the statistical analysis tool to formulate recommendations for students and companies.

[8] "Institutional email" stand for the email assigned to the student by their university.

• [R12] The system shall provide suggestions to companies members for the project description.

• [R13] The system shall provide suggestions to students regarding their CVs.

• [R14] The system shall allow the company members to select the students, among all candidates, for the internship.

• [R15] The system must collect feedbacks from students and companies members in order to keep statistics up-to-date.

• [R16] The system must send feedbacks to the statistical analysis tool in order to update its analysis.

• [R17] The system shall allow the company member to schedule a meeting with a candidate.

• [R18] The system must allow the student, the company members and the university members to monitor the ongoing internship.[9]

• [R19] The system must allow interrupting an internship when university members requires it.[10]

• [R20] The system must provide to student all available internships related to the keyword specified by him.

• [R21] The system shall inform the company members and the student that the internship has concluded cause of the decision of the student's university.

• [R22] The system must prevent students to apply for the internship after closing the application period.

• [R23] The system shall allow students to modify their CV.

• [R24] The system shall allow companies members to modify internship description.

• [R25] The system must allow students and companies members (that are involved in the same internship) to send messages to each other during the selection of the candidates.

• [R26] The system must allow students and companies members to posts reports regarding internship.

• [R27] The system must notify the company members about the new matching proposal made by the student.

• [R28] The system shall allow the student to send a matching proposal to company members.

• [R29] The system shall allow company members to send a matching proposal to students.

• [R30] The system must inform the student about a new matching proposal made by the company members.

[9] The monitoring is achieved through report visualization.

[10] Only university members could interrupt the internship and not company members.

• [R31] The system must inform the student or the company member of the outcome of the matching proposal he sent to the counterpart.

• [R32] The system shall allow the company members or the student to either accept or decline the matching proposal made by the counterpart.

• [R33] The system must allow company members to delete an internship only when it is closed for applications.

• [R34] The system must enable students to accept or decline offers after being selected to start the internship.

• [R35] The system must allow students to post complaints.

3.3 Use cases

3.3.1 Use case diagrams

Below there is the use cases diagram. For clarity, it has been divided into multiple diagrams.

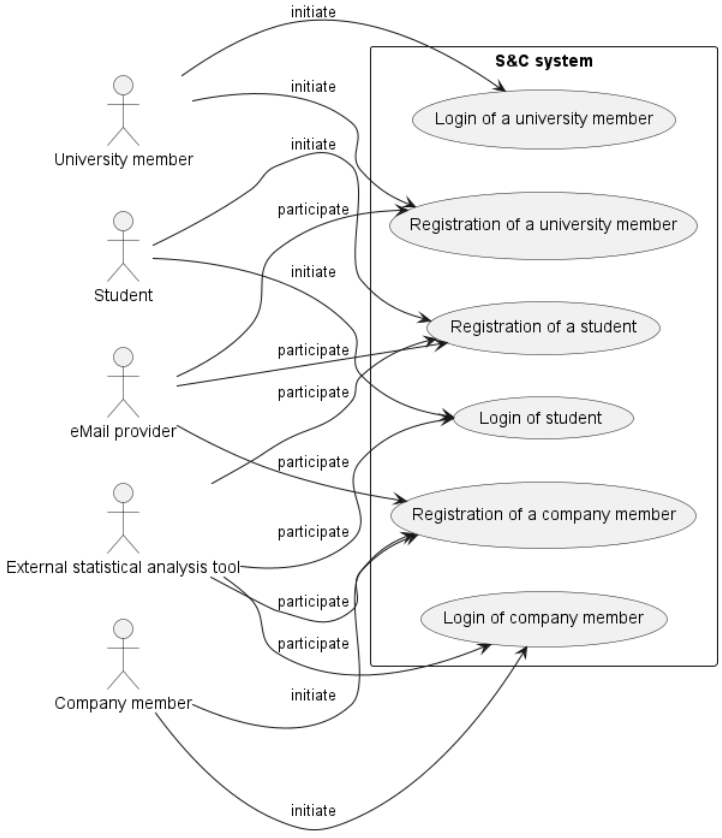

Figure 5: Use cases from UC1 to UC6.

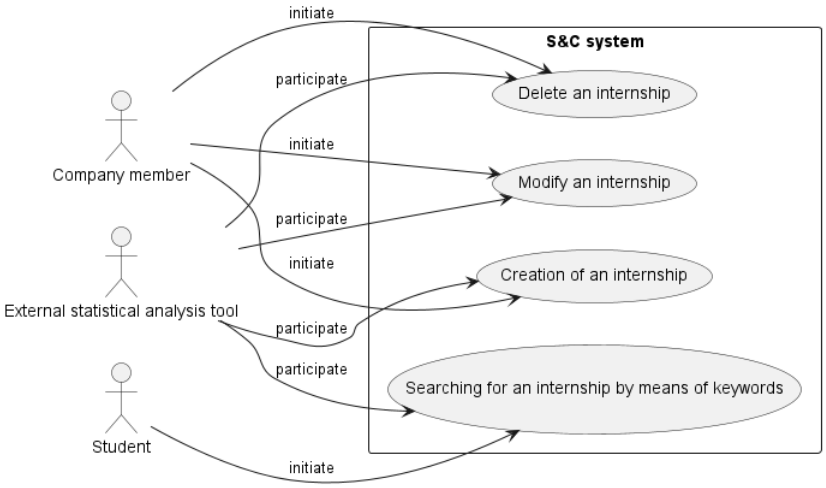

Figure 6: Use cases from UC7 to UC10.

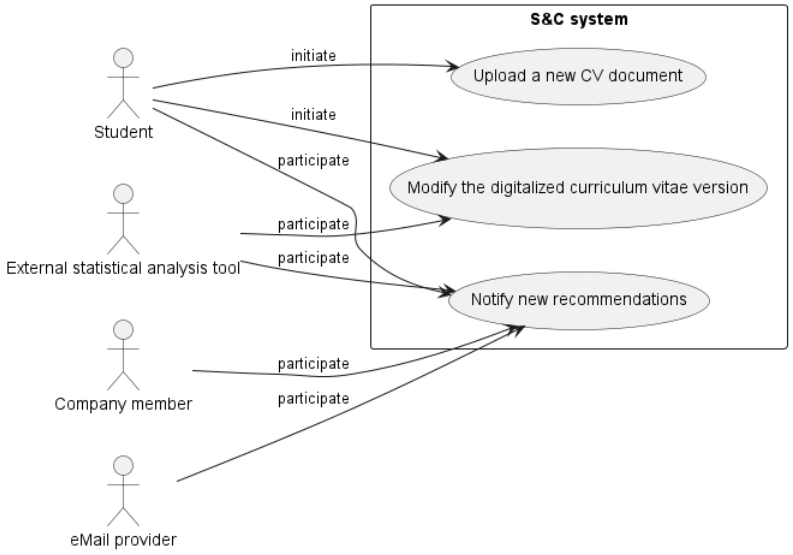

Figure 7: Use cases from UC11 to UC13.

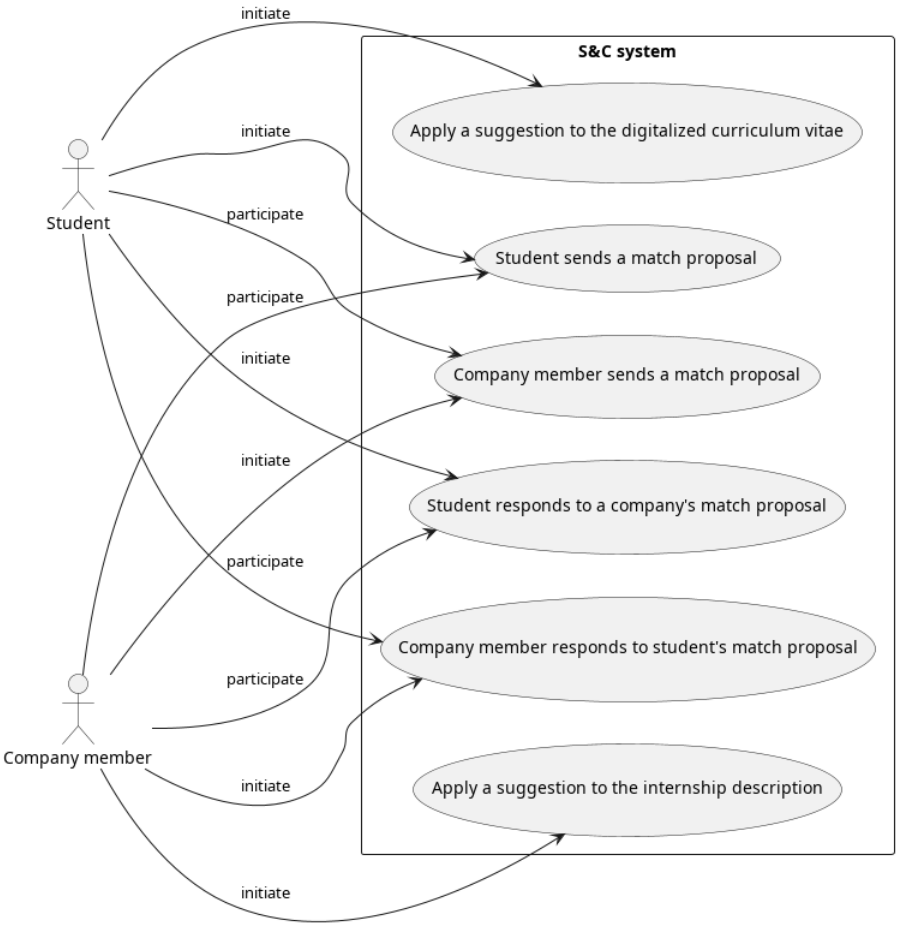

Figure 8: Use cases from UC14 to UC19.

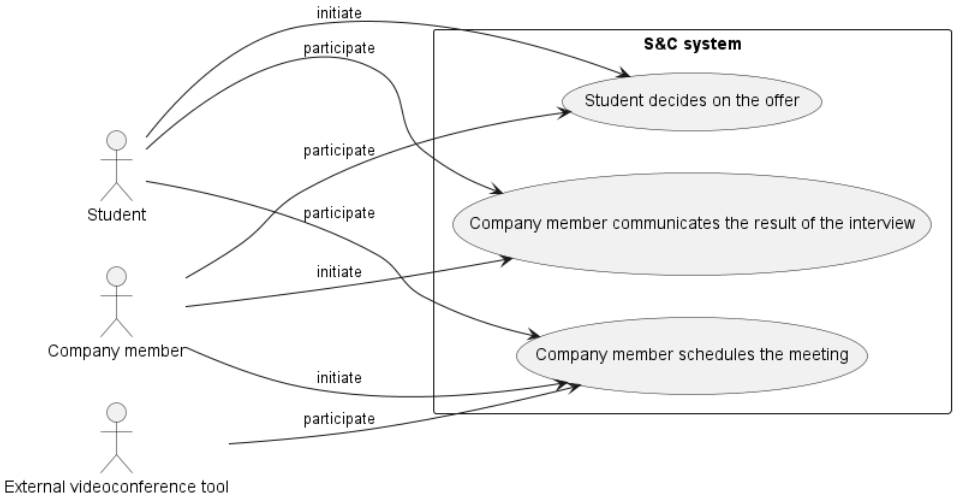

Figure 9: Use cases from UC20 to UC22.

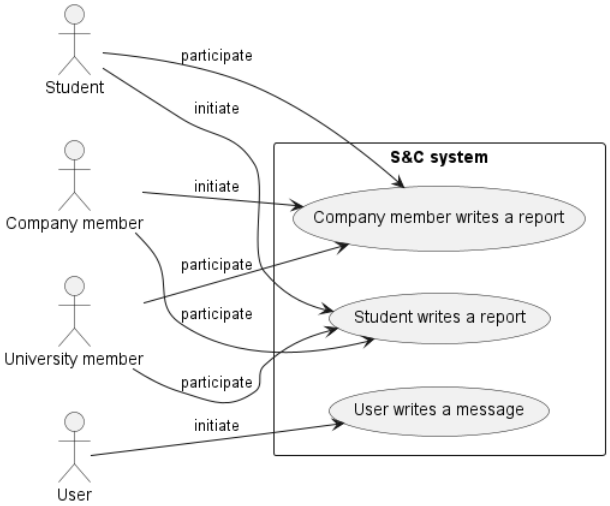

Figure 10: Use cases from UC23 to UC25.

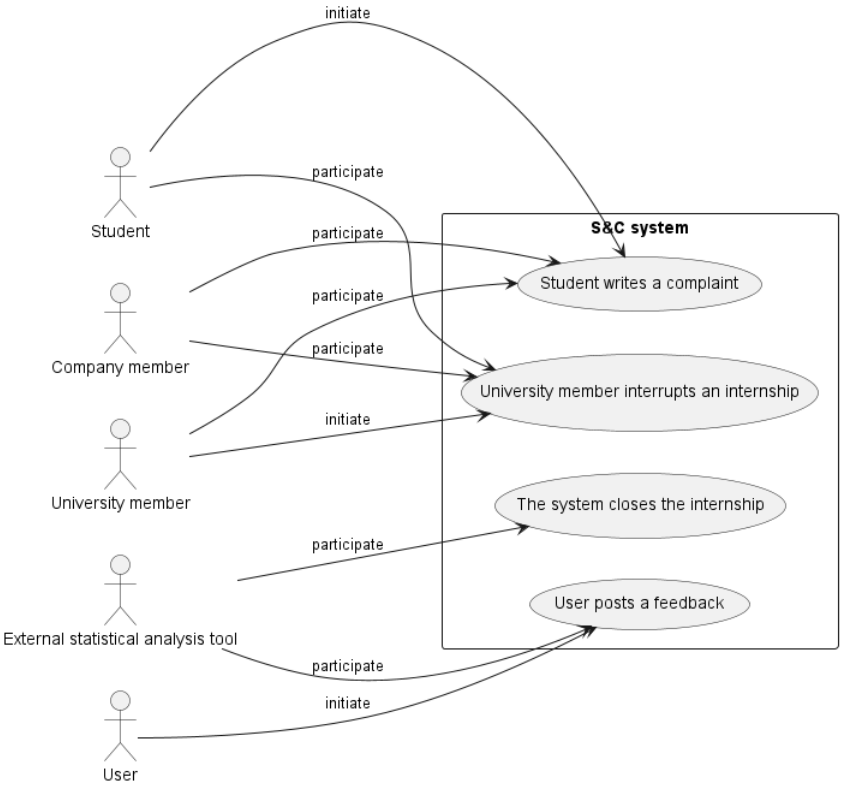

Figure 11: Use cases from UC26 to UC29.

3.3.2 Use cases in details

[UC1] - Registration of a university member

Name	Registration of a university member
Actors	• University member • EMail provider
Entry Condition	The university member clicks on button *"Register as a university member"*.
Event Flow	1. The system shows the *"Registration form"*. 2. The university member enters his name, surname and date of birth, a password and his institutional email. 3. The university member selects his university amoung all the universities registered to the system. 4. The university member clicks the button *"Register"*. 5. The system verifies that the email is associated to the university provided. 6. The system verifies that the email is not already registered to the system. 7. The system sends an email to the university member by an eMail provider.
Exit Condition	The university member verifies the email provided to the system by following the instructions on the email message that he received.

Exceptions

- The email is already registered to the system. The system will show an error message *"User already registered"* and it will interrupt the registration process.
- The email provided is not linked to the university provided by the member. In this case the system will show an error message *"Invalid email domain"*.
- The university in which the university member works is not present in the system. In this case the university member clicks the button *"Create your university"* and by clicking it the system will show the *"University registration form"* in which the university member will enter all the information regarding the university. Once he has confirmed all data, the institution will be inserted to the system and moreover it will sends a confirmation email to the university member.
- The member does not confirm their email address within one day of entering the data into the system. In this case the system will delete all provided data. The member will then need to repeat the registration process.

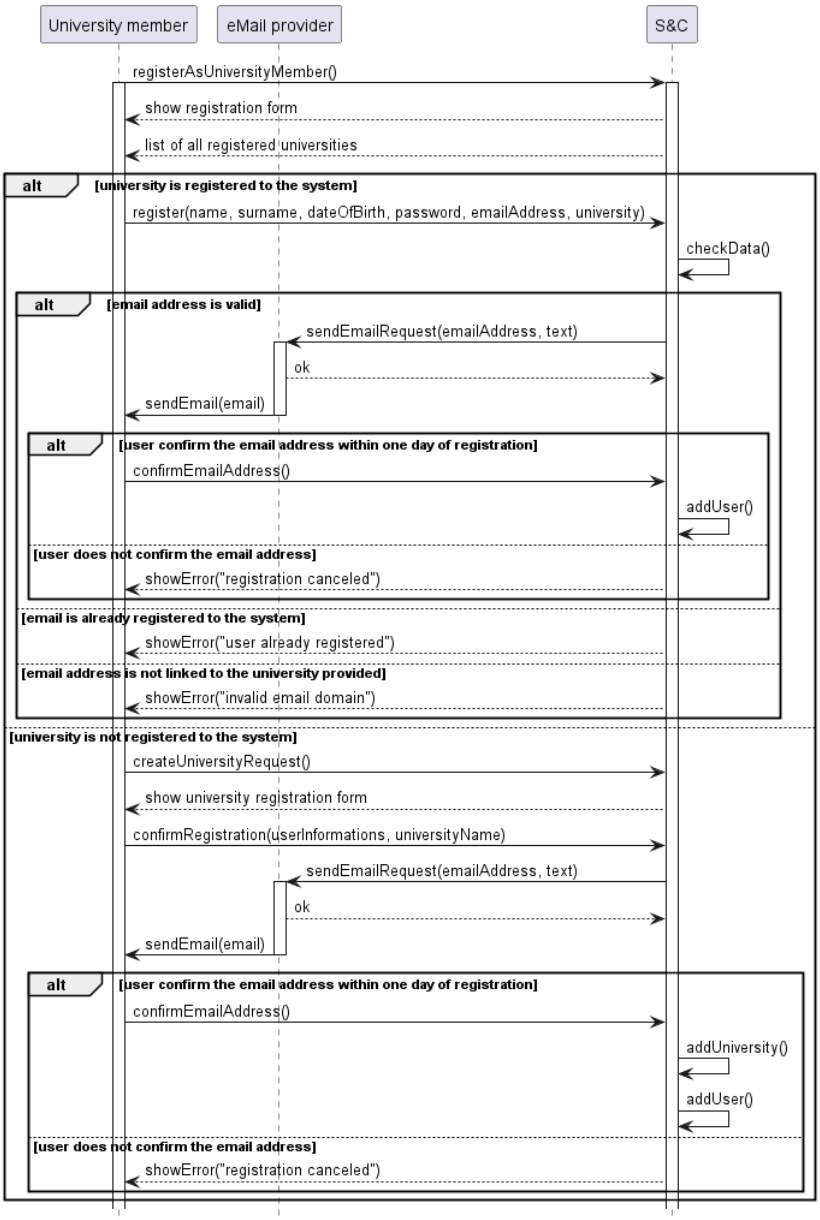

Figure 12: Sequence diagram of *"Registration of a university member"* use case.

[UC2] - Registration of a company member

Name	Registration of a company member
Actors	• Company member • External statistical analysis tool • eMail provider
Entry Condition	The company member clicks on button *"Register as a company member"*.
Event Flow	1. The system shows the *"Registration form.* 2. The company member enters his name, surname and date of birth, a password and his institutional email. 3. The company member selects his company amoung all the companies registered to the system. 4. The company member clicks the button *"Register"*. 5. The system verifies that the email is associated to the company provided. 6. The system verifies that the email is not already registered to the system. 7. The system sends an email to the company member by an eMail provider.
Exit Condition	The company member verifies the email provided to the system by following the instructions on the email message that he received.

Exceptions

- The email is already registered to the system. The system will show an error message *"User already registered"* and it will interrupt the registration process.
- The email provided is not linked to the company provided by the company member. In this case the system will show an error message *"Invalid email domain"*.
- The company in which the company member works is not present in the system. In this case the comapany member clicks the button *"Create your company"* and by clicking it the system will show the *"Company registration form"* in which the company member will enter all the information regarding the company. Once he has confirmed all data, the institution will be inserted to the system and moreover it will sends a confirmation email to the company member.
- The company member does not confirm their email address within one day of entering the data into the system. In this case the system will delete all provided data. The company member will then need to repeat the registration process.

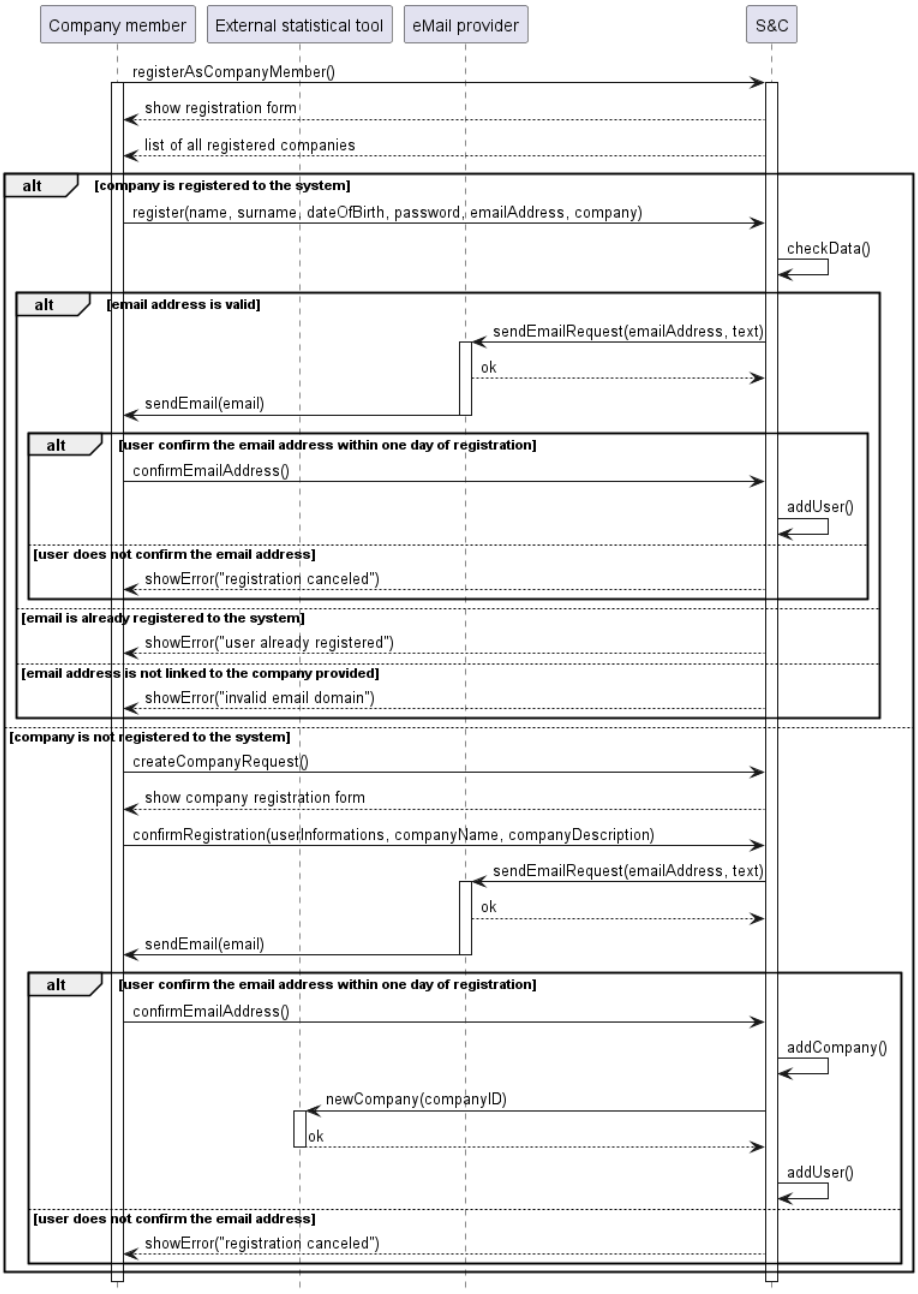

Figure 13: Sequence diagram of *"Registration of a company member"* use case.

[UC3] - Registration of a student

Name	Registration of a student
Actors	• Student • External statistical analysis tool • eMail provider
Entry Condition	The student clicks on button *"Register as a student"*.
Event Flow	1. The system shows the *"Registration form"*. 2. The student enters his name, surname and date of birth, a password and his institutional email. 3. The student selects his university amoung all the universities registered to the system. 4. The member clicks the button *"Register"*. 5. The system verifies that the email is associated to the university provided. 6. The system verifies that the email is not already registered to the system. 7. The system sends an email to the student by an eMail provider. 8. The student verifies the email provided to the system by following the instructions on the email message that he received.
Exit Condition	The system sends the identifier of the student newly created to the external statistical analysis tool.

Exceptions

- The email is already registered to the system. The system will show an error message *"User already registered"* and it will interrupt the registration process.
- The email provided is not linked to the university provided by the student. In this case the system will show an error message *"Invalid email domain"*.
- The student does not confirm their email address within one day of entering the data into the system. In this case the system will delete all provided data. The student will then need to repeat the registration process.

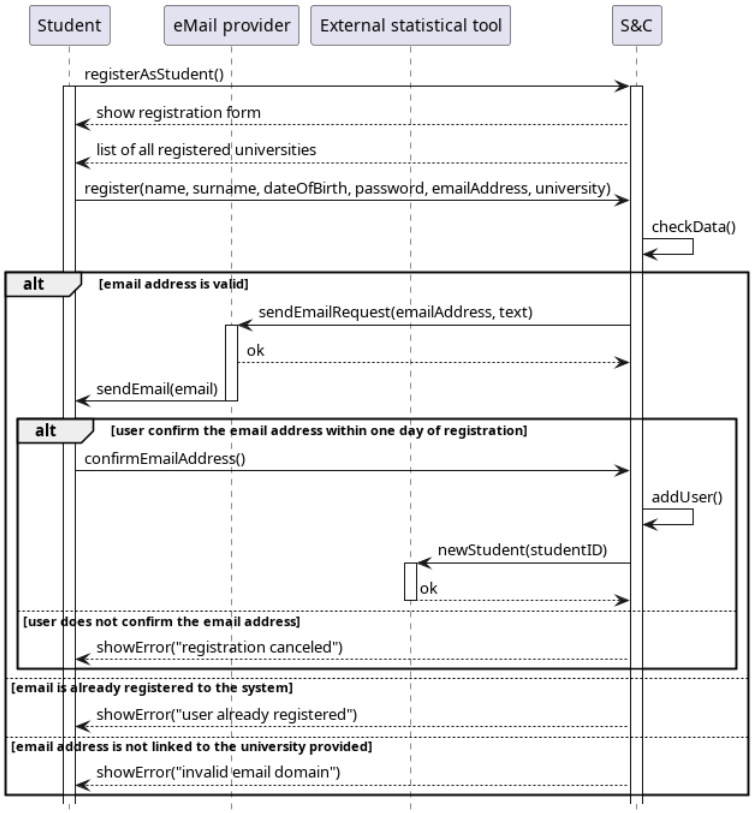

Figure 14: Sequence diagram of *"Registration of a student"* use case.

[UC4] - Login of a university member

Name	Login of a university member
Actors	University member
Entry Condition	The university member clicks on button *"Login"*.
Event Flow	1. The system shows the *"Login form"*. 2. The university member inserts his credentials (email and password). 3. The university member clicks on button *"Confirm"*. 4. The system checks credentials.
Exit Condition	The system shows the *"Homepage"* of the application.
Exceptions	Credentials are invalid. In this case the system will show an error message *"Invalid credentials"* and it will clear the input fields.

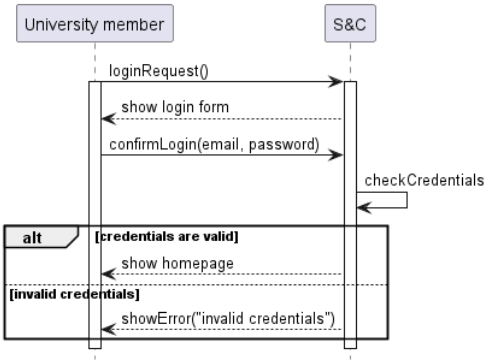

Figure 15: Sequence diagram of *"Login of a university member"* use case.

[UC5] - Login of student

Name	Login of student
Actors	• Student • External statistical analysis tool
Entry Condition	The student clicks on button *"Login"*.
Event Flow	1. The system shows the *"Login form"*. 2. The user inserts his credentials (email and password). 3. The user clicks on button *"Confirm"*. 4. The system checks credentials. 5. The system requests all the recommendations associated with the student from the external statistical analysis tool. 6. The system receives the recommendations.
Exit Condition	The system displays the application's *"homepage"* containing the recommendations found.
Exceptions	Credentials are invalid. In this case the system will show an error message *"Invalid credentials"* and it will clear the input fields.

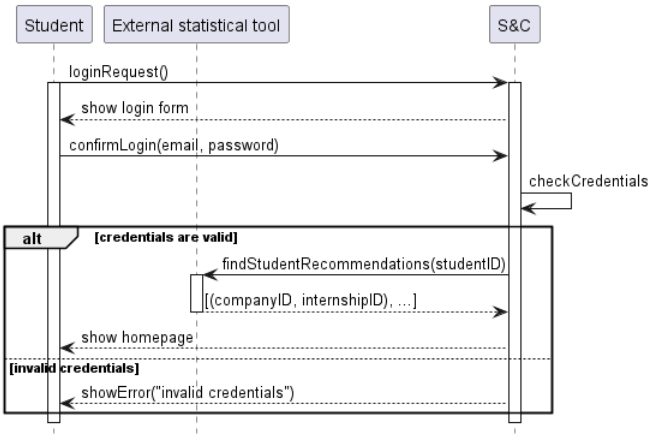

Figure 16: Sequence diagram of *"Login of student"* use case.

[UC6] - Login of company member

Name	Login of company member
Actors	• Company member • External statistical analysis tool
Entry Condition	The company member clicks on button *"Login"*.
Event Flow	1. The system shows the *"Login form"*. 2. The company member inserts his credentials (email and password). 3. The company member clicks on button *"Confirm"*. 4. The system checks credentials. 5. The system requests all the recommendations associated with the company of the member from the external statistical analysis tool. 6. The system receives the recommendations.
Exit Condition	The system displays the application's *"homepage"* containing the recommendations found.
Exceptions	Credentials are invalid. In this case the system will show an error message *"Invalid credentials"* and it will clear the input fields.

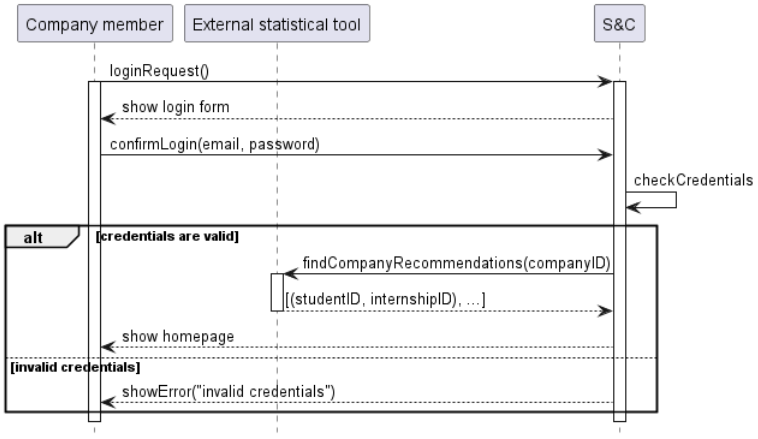

Figure 17: Sequence diagram of *"Login of company member"* use case.

[UC7] - Creation of an internship

Name	Creation of an internship
Actors	• Company member • External statistical analysis tool
Entry Condition	The company member is logged to the platform and he press the button *"Create an internship"*.
Event Flow	1. The system shows the *"Internship form"*. 2. The company member enters all the required information: a name, the valid application period, the maximum number of students allowed, the description, the requirements, the domain of application, tags and the terms specified in the contract. 3. The company member clicks on button *"Confirm"*. 4. The system sends the internship identifier and all the internship tags to the external statistical analysis tool.
Exit Condition	The system will redirect the company member to the details section related to the internship just created.
Exceptions	

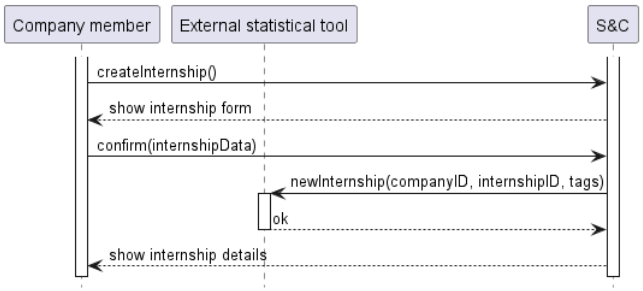

Figure 18: Sequence diagram of *"Creation of an internship"* use case.

[UC8] - Modify an internship

Name	Modify an internship
Actors	• Company member • External statistical analysis tool
Entry Condition	The company member is logged in the system and he is on the *"Internships list"* section. He clicks on the button *"Modify the internship"* near a specific internship. The selected internship is closed for applications and there are not enorelled students.
Event Flow	1. The system displays the form to allow data modification. 2. The company member edits the information. 3. The company member clicks the button *"See modifications"*. 4. The system shows to the company member all modified fields. 5. The company member clicks on button *"Confirm all the changes"*. 6. The system sends the new set of tags related to the modified internship to the external statistical analysis tool.
Exit Condition	The system shows the internship page with all the updates.
Exceptions	• The company member does not confirm changes. In this case the system will stop the procedure.

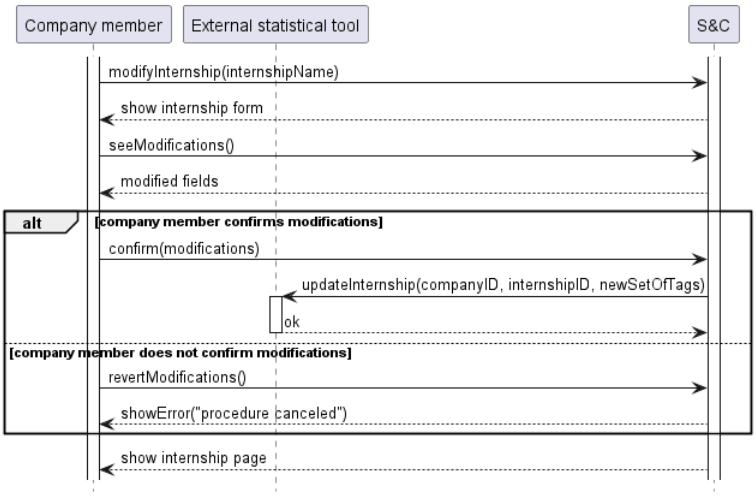

Figure 19: Sequence diagram of *"Modify an internship"* use case.

[UC9] - Delete an internship

Name	Delete an internship
Actors	• Company member • External statistical analysis tool
Entry Condition	The company member is logged in the system and he is on the *"Internships list"* section. He clicks on the button *"Delete the internship"* near a specific internship. The selected internship is closed for applications and there are not enorelled students.
Event Flow	1. The system displays a banner to the company member to confirm the deletion of the internship. 2. The company member clicks on the button *"Confirm deletion"*. 3. The system deletes the internship. 4. The system notifies the external statistical analysis tool of the deletion.
Exit Condition	The system shows the message *"Deletion performed"* and redirects the company member to the *"Internships list"* section.
Exceptions	The company member does not confirm the deletion. In this case the system will show the *"Internships list"* section.

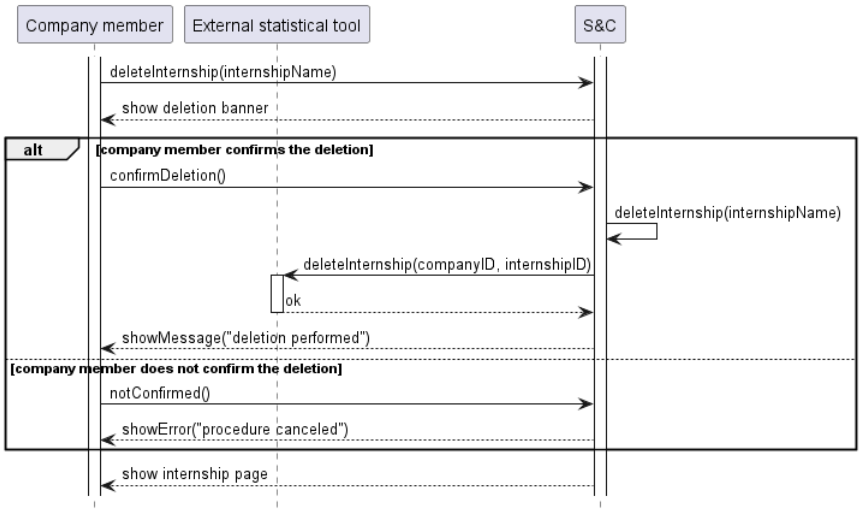

Figure 20: Sequence diagram of *"Delete an internship"* use case.

[UC10] - Searching for an internship by means of keywords

Name	Searching for an internship by means of keywords
Actors	• Student • External statistical analysis tool
Entry Condition	The student is logged in and enters keywords into the *"Internship search bar"* located on the homepage of the platform.
Event Flow	1. The student clicks on the *"Confirm"* button next to the search bar. 2. The system requests all recommendations associated with the student from the external statistical analysis tool. 3. The external statistical analysis tool sends all the retrieved recommendations to the system. 4. The system gathers all internships that have a common part in their name, tags, or description with the entered text, ordering them based on the retrieved recommendations.
Exit Condition	The system shows to the student all the results
Exceptions	The system does not find any results related to the entered keywords. The system will shows the message *"No internships found"*.

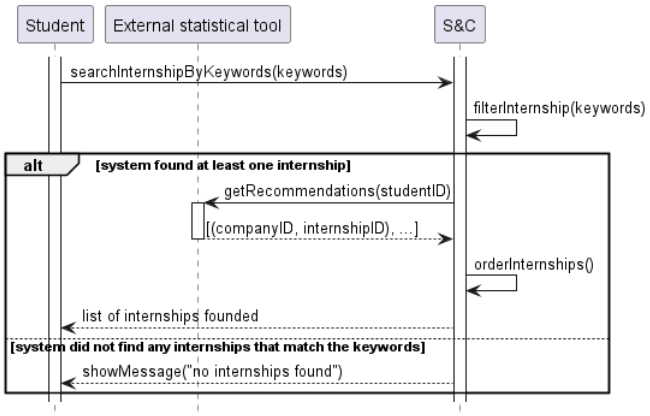

Figure 21: Sequence diagram of *"Searching for an internship by means of keywords"* use case.

[UC11] - Modify the digitalized curriculum vitae version

Name	Modify the digitalized curriculum vitae version
Actors	• Student • External statistical analysis tool
Entry Condition	The student is logged in and he clicks the button *"Modify your digitalized curriculum"* in their personal page.
Event Flow	1. The system shows the *"Modification form"*. This form includes all the sections added by the student. 2. The student clicks on button *"Modify section"* to modify a specific section of his curriculum. 3. The student inserts all the modifications. 4. The student clicks on button *"Save changes"*. 5. The system sends information about tag modifications to the external analysis tool.
Exit Condition	The system shows to the student the new version of the digitalized curriculum.
Exceptions	

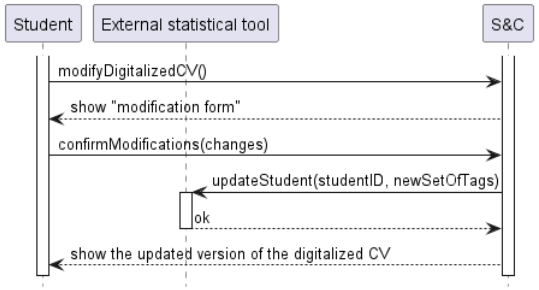

Figure 22: Sequence diagram of *"Modify the digitalized curriculum vitae version"* use case.

[UC12] - Upload a new CV document

Name	Upload a new CV document
Actors	Student
Entry Condition	The student is loggeg in. He is on the personal page and he clicks the button *"Upload a new version of your curriculum"*.
Event Flow	1. The system displays a window showing the files saved on the student's device. 2. The student selects the file to be uploaded in the system. 3. The system stores the new file.
Exit Condition	The system shows to the student his personal page.
Exceptions	The file selected by the student has an invalid file extension. In this case, the system will send an error message, *"Invalid file extension"*, to the student.

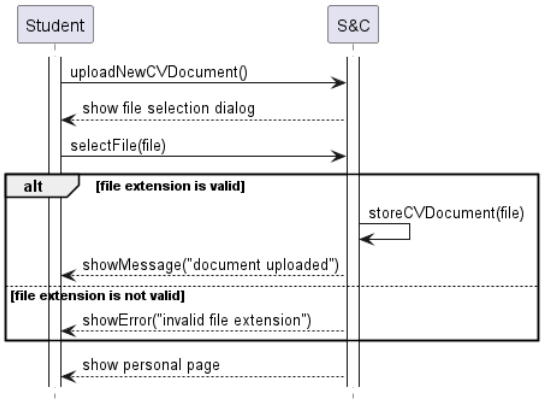

Figure 23: Sequence diagram of *"Upload a new CV document"* use case.

[UC13] - Notify new recommendations.

Name	Notify new recommendations.
Actors	• External statistical analysis tool • Student • Company member • eMail provider.
Entry Condition	The timer to notify the recommendations has ended.
Event Flow	1. The system requests the recommendations of registered students and company members from the external statistical analysis tool. 2. The system receives all the recommendations.
Exit Condition	The system notifies all students and company members involved in at least one recommendation by sending them an email (using the eMail provider).
Exceptions	

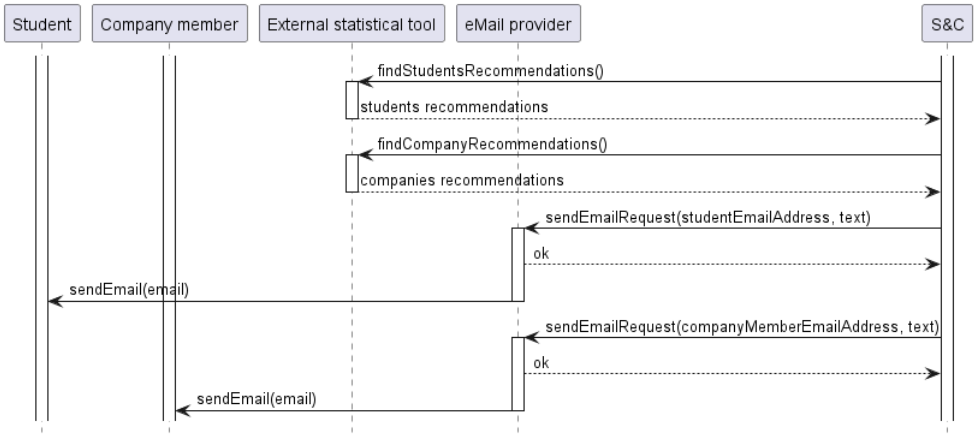

Figure 24: Sequence diagram of *"Notify new recommendations."* use case.

[UC14] - Apply a suggestion to the digitalized curriculum vitae

Name	Apply a suggestion to the digitalized curriculum vitae
Actors	Student
Entry Condition	The system detects missing information in the curriculum after the student makes changes to it.
Event Flow	1. The system shows a banner on the homepage that suggest to student to add the missing information. 2. The student clicks on the banner. 3. The system shows to the student personal page with his digitalized curriculum in edit mode. 4. The student add missing information.
Exit Condition	The student save modifications.
Exceptions	The student close his personal page befor inserting missing data. The system will still prompt the student to provide information during the next access.

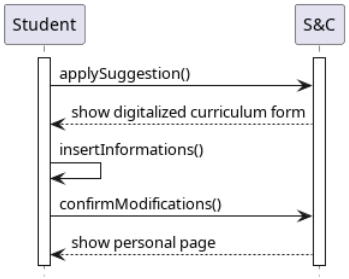

Figure 25: Sequence diagram of *"Apply a suggestion to the digitalized curriculum vitae"* use case.

[UC15] - Apply a suggestion to the internship description

Name	Apply a suggestion to the internship description
Actors	Company member
Entry Condition	The system detects missing information in the internship description after the company member makes changes to it.
Event Flow	1. The system shows a banner on the homepage that suggest to the company member to add the missing information on a specific internship. 2. The company member clicks on the banner. 3. The system shows to the company member the internship overview in edit mode. 4. The company member add missing information.
Exit Condition	The company member save modifications.
Exceptions	The company member close the internship page befor inserting missing data. The system will still prompt the company member to provide information during the next access.

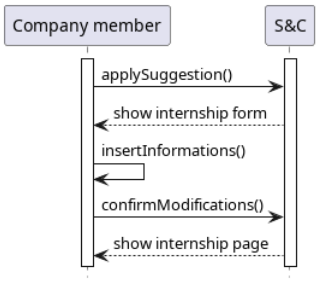

Figure 26: Sequence diagram of *"Apply a suggestion to the internship description"* use case.

[UC16] - Student sends a match proposal

Name	Student sends a match proposal
Actors	• Student • Company member
Entry Condition	The student is logged into the platform's homepage and intends to apply for an internship.
Event Flow	1. The student selects an internship from the system's suggestions. 2. The system redirects the student to the internship details page. 3. The student clicks the *"Send Request"* button to apply. 4. The system shows the request has arrived.
Exit Condition	The company has the student's request.
Exceptions	• The student clicks the *"Send request"* button the second time. The system shows an error. • The internship is closed (the application period is over), the system shows an error.

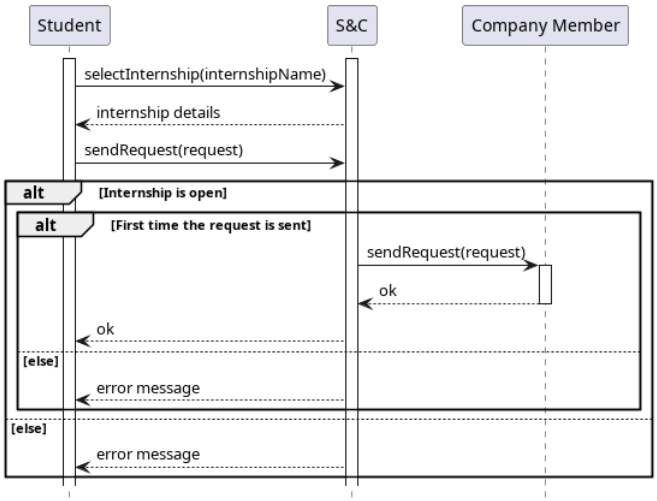

Figure 27: Sequence diagram of *"Student sends a match proposal"* use case.

[UC17] - Company member sends a match proposal

Name	Company member sends a match proposal
Actors	• Student • Company member
Entry Condition	The company member is logged into the platform's homepage and intends to requests a student for an opened internship
Event Flow	1. The company member selects a student profile suggested by the system. 2. The system redirects the company member to the student's profile page. 3. The company member clicks the *"Send Request"* button to send an invitation. 4. If the list of candidate for the internship has not yet been created, the system creates one. 5. The system adds the student to the list of candidate. 6. The system shows the request has arrived to the student.
Exit Condition	The student has the company's invitation.
Exceptions	The company member clicks the *"Send request"* button the second time. The system shows an error.

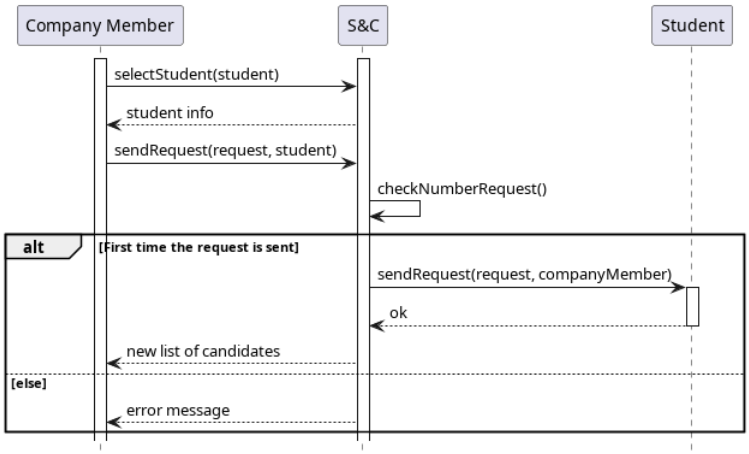

Figure 28: Sequence diagram of *"Company member sends a match proposal"* use case.

[UC18] - Student responds to company's match proposal

Name	Student responds to company's match proposal
Actors	• Student • Company member
Entry Condition	The student is logged into the platform's homepage where the company's request is visible.
Event Flow	1. The student views the proposal from the company and selects either the *"Accept"* or *"Refuse"* button. 2. The system verifies if the internship is still available. 3. The system notifies the recruiter about the student's decision. 4. If accepted, the system initiates a private chat between the student and the recruiter for further communication.
Exit Condition	The system has successfully sent the notification to the company member.
Exceptions	The internship is closed. An error message appears on the screen and the system automatically redirects the student to the homepage.

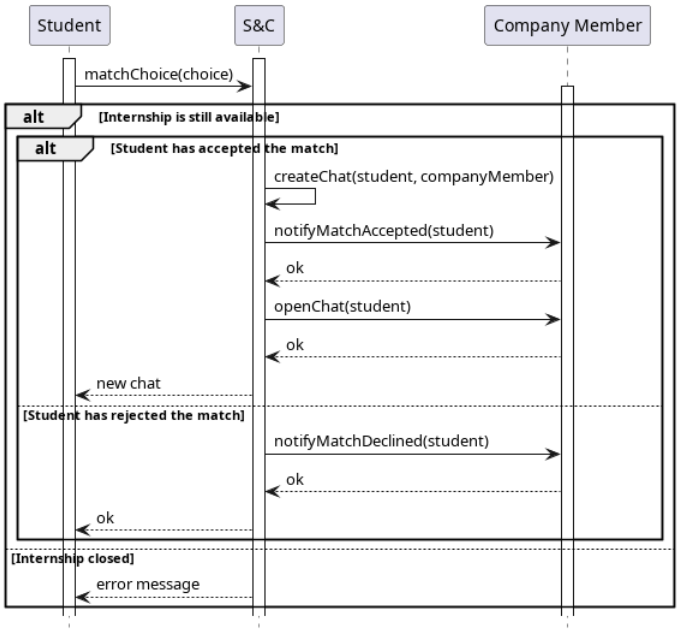

Figure 29: Sequence diagram of *"Student responds to company's match proposal"* use case.

[UC19] - Company member responds to student's match proposal

Name	Company member responds to student's match proposal
Actors	• Company member • Student
Entry Condition	The company member is logged in the platform's homepage where the student's request is visible.
Event Flow	1. The company member reviews the student's request and selects either the *"Accept"* or *"Refuse"* button. 2. The system notifies the student of the decision. 3. If the student's request is accepted, the candidate list is created (if none exists) and the student is added to it.
Exit Condition	The notification arrives to the student. If the student has been accepted, a private chat is created.
Exceptions	

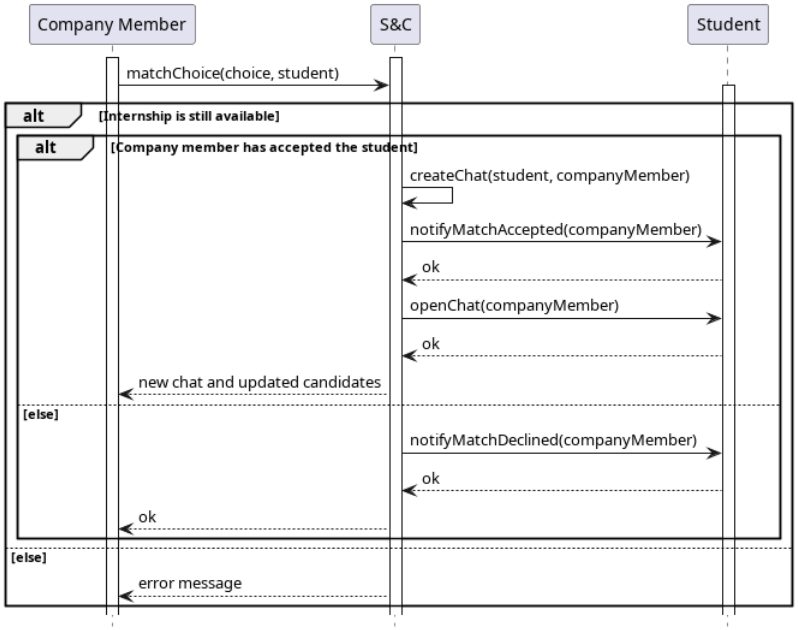

Figure 30: Sequence diagram of *"Company member responds to student's match proposal"* use case.

[UC20] - Company member schedules the meeting

Name	Company member schedules the meeting
Actors	• Company member • Student • External videoconference tool
Entry Condition	The company member and the student are correctly logged in with their chat opened. They agree on the meeting time and where it will be held.
Event Flow	1. The company member clicks the *"Schedule a Meeting"* button. 2. The system displays a form for meeting details (e.g., date, time, location, purpose). 3. The company member fills in the required details and confirms the entry. 4. The system notifies the student of the scheduled meeting. 5. The system adds the meeting to the interview reminder of both the company member and the student. If the meeting is online, the system contacts an external tool to generate a link for the video conference and attaches it to the event.
Exit Condition	The meeting is in the student's and company member's reminder.
Exceptions	• The system identifies errors in the scheduled meeting (e.g illegal time frame) and shows a message to the screen. • The system detects an overlap with an event already in the interview reminders and shows a warning to the user.

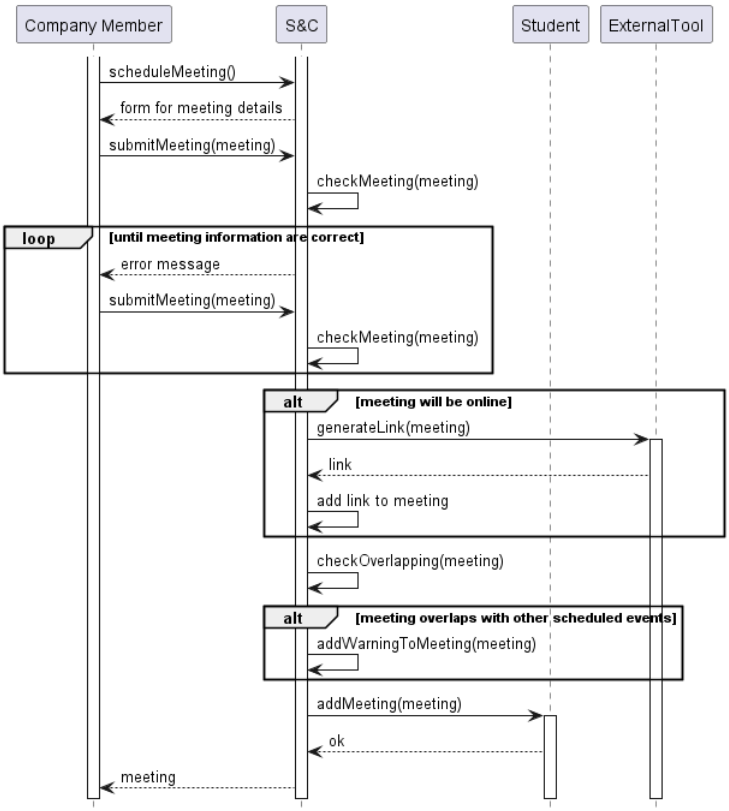

Figure 31: Sequence diagram of *"Company member schedules the meeting"* use case.

[UC21] - Company member communicates the result of the interview

Name	Company member communicates the result of the interview
Actors	• Company member • Student
Entry Condition	The company member is logged into the platform with the interview process result finalized.
Event Flow	1. The recruiter selects a student from the list of candidates. 2. The system displays the student's profile. 3. The recruiter clicks either the *"Accept"* or *"Reject"* button to finalize the decision. 4. The system notifies the student of the decision. If accepted, the system attaches the contract specified in the internship offer; otherwise, the private chat is deleted.
Exit Condition	The notification arrives to the student. If the student is rejected, the chat is no more available: any access will show an error message.
Exceptions	

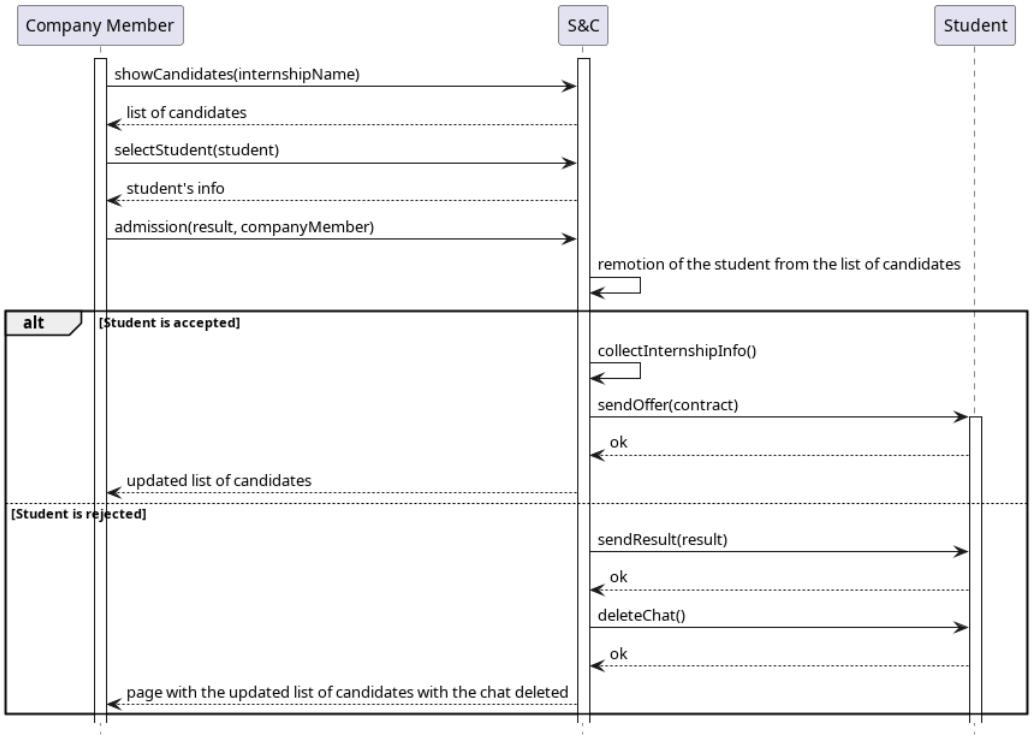

Figure 32: Sequence diagram of *"Company member communicates the result of the interview"* use case.

[UC22] - Student decides on the offer

Name	Student decides on the offer
Actors	• Company member • Student
Entry Condition	The student is logged into the platform's homepage. The notification of the acceptance from the company has arrived.
Event Flow	1. The student clicks on the notification about the company member's decision. 2. The system displays the decision along with *"Accept"* and *"Refuse"* buttons. 3. The student selects their choice by clicking one of the buttons. 4. The system notifies the company member of the student's response. 5. The private chat is deleted.
Exit Condition	The notification arrives to the company member. The chat is no more available.
Exceptions	

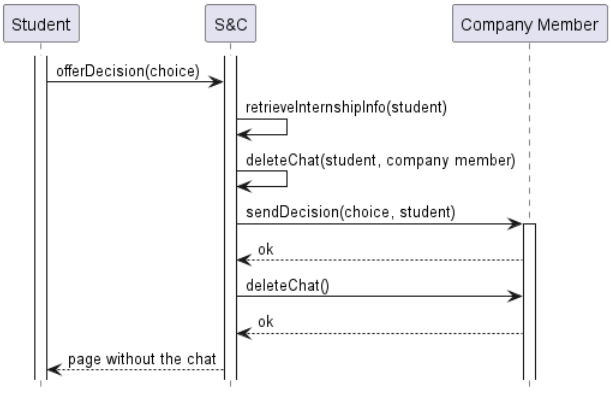

Figure 33: Sequence diagram of *"Student decides on the offer"* use case.

[UC23] - User writes a message

Name	User writes a message
Actors	User
Entry Condition	User is logged into the platform's homepage and knows the message to write along with the recipient's information
Event Flow	1. The user clicks on the chat icon to open the messaging interface. 2. The system displays a text box for the message. 3. The user composes the message. 4. The user selects the intended recipient. 5. The user clicks the *"Submit"* button to send the message. 6. The system adds the message to the sender's and recipient's chat.
Exit Condition	The message is part of the sender's and recipient's chat.
Exceptions	The message is empty or no recipient is specified, the system shows an error message.

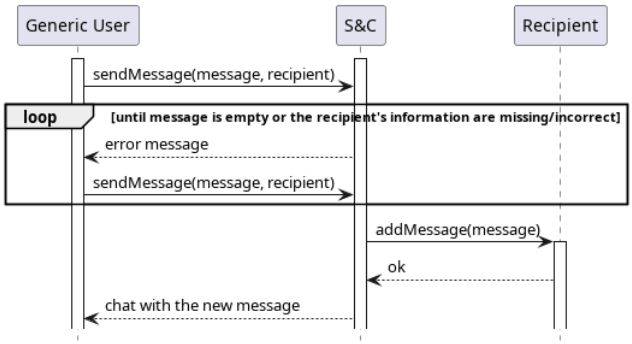

Figure 34: Sequence diagram of *"User writes a message"* use case.

[UC24] - Student writes a report

Name	Student writes a report
Actors	• Student • Company member • University member
Entry Condition	The student is logged into the platform's homepage
Event Flow	1. The student navigates to the list of reports. 2. The student clicks the *"Create a New Report"* button. 3. The system displays a text box for the student to compose the report. 4. The student completes and submits the report. 5. The system automatically adds the student's name and surname to the report. 6. The report is shared with all relevant parties, including the student, the university, and the company members.
Exit Condition	The report is added to the list of all interested users.
Exceptions	The report is empty. The system displays an error message.

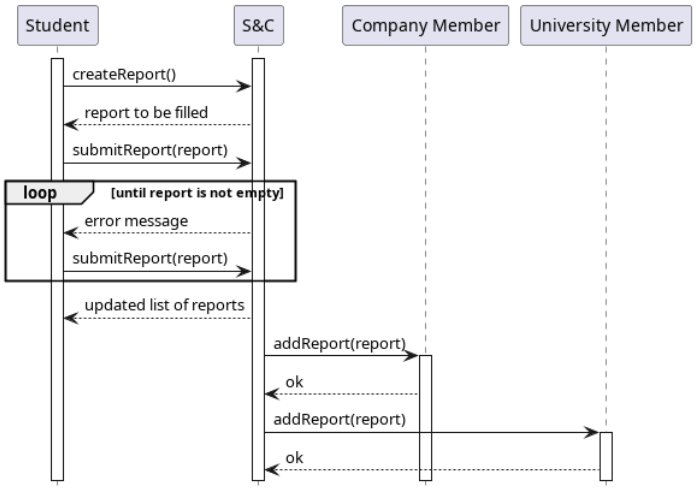

Figure 35: Sequence diagram of *"Student writes a report"* use case.

[UC25] - Company member writes a report

Name	Company member writes a report
Actors	• Company member • Student • University member
Entry Condition	The company member is logged into the platform's homepage
Event Flow	1. The company member navigates to the list of reports. 2. The company member clicks the *"Create a Report"* button. 3. The system displays a text box for composing the report. 4. The company member writes the report and selects the student it pertains to. 5. The company member submits the report. 6. The system automatically adds the company member's name and surname to the report. 7. The system shares the report with relevant parties, including the selected student and their university.
Exit Condition	The report is added to the list of all interested users.
Exceptions	• No student has been inserted or they are not woking at Code S.P.A, the system shows an error message. • The report is empty, the system shows an error message.

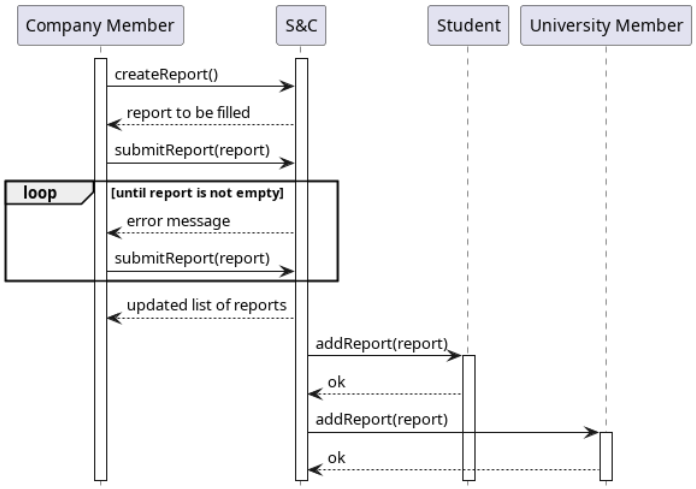

Figure 36: Sequence diagram of *"Company member writes a report"* use case.

[UC26] - Student writes a complaint

Name	Student writes a complaint
Actors	• Student • University member • Company member
Entry Condition	The student is in the homepage.
Event Flow	1. The student clicks the *"Write Complaint"* button. 2. The system displays a text box for the complaint. 3. The student writes the complaint and submits it. 4. The system attaches the student's information to the complaint and adds it to the list of complaints of all users responsible for the internship.
Exit Condition	The complaint is added to the list of all interested parties.
Exceptions	The complaint is empty. The system shows an error message.

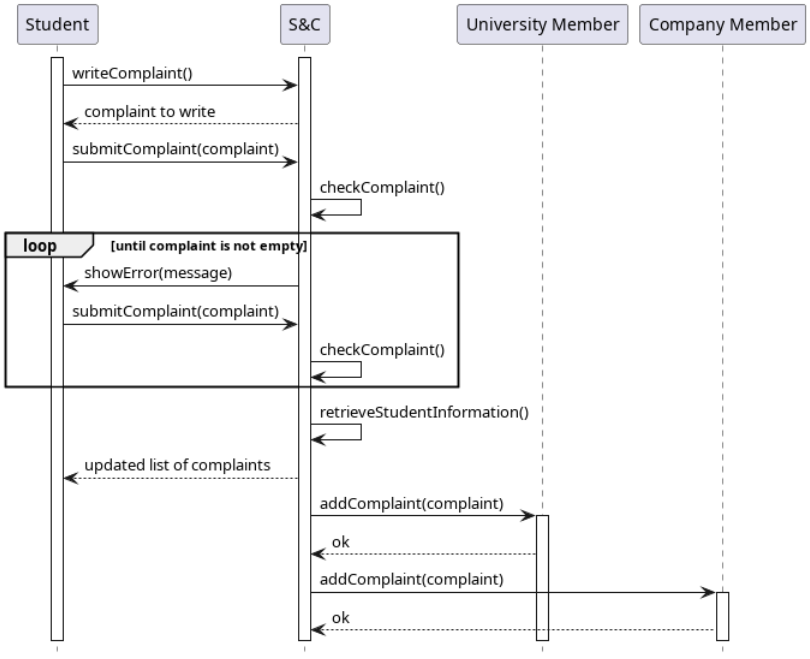

Figure 37: Sequence diagram of *"Student writes a complaint"* use case.

[UC27] - University member interrupts an internship

Name	University member interrupts an internship
Actors	• University member • Company member • Student
Entry Condition	University member is logged into the platform's homepage.
Event Flow	1. The university member clicks the *"Manage Students"* button. 2. The system displays a list of students currently participating in internships. 3. The university member selects a student and clicks the *"Interrupt Internship"* button. 4. The system notifies all involved parties, including the student and the company members, about the interruption. 5. The system updates the internship page to display the message *"Interrupted Internship"*.
Exit Condition	All users involved received the notification regarding the interruption of the internship. Any access to the page related to the internship will show the message *"Interrupted Internship"*.
Exceptions	

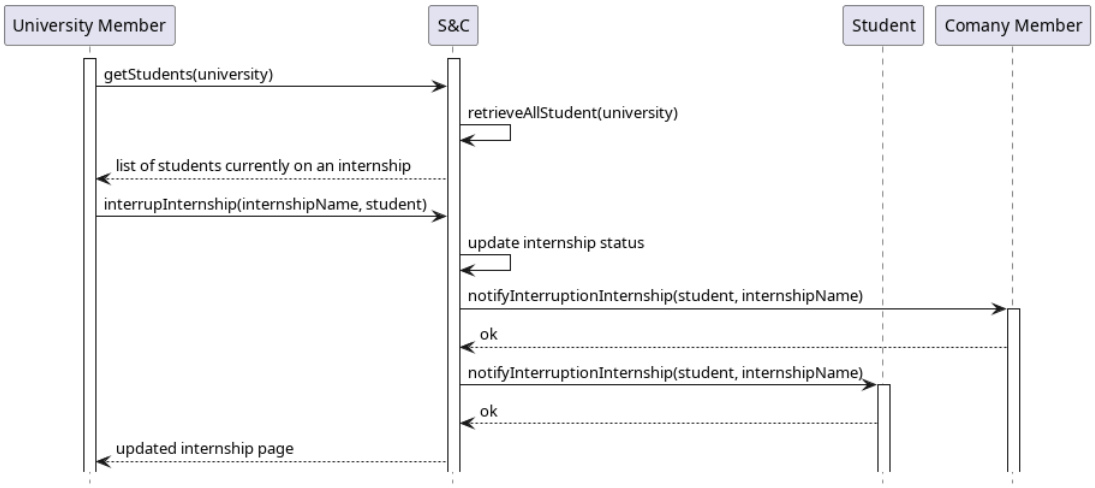

Figure 38: Sequence diagram of *"University member interrupts an internship"* use case.

[UC28] - Student or company member posts a feedback

Name	Student or company member posts a feedback
Actors	• Student • Company member • External statistical analysis tool
Entry Condition	The student or company member is logged into the platform's homepage. The internship is ended.
Event Flow	1. The user notices the system notification indicating the end of the internship. 2. The user clicks on the notification. 3. The system prompts the user with the message *"Would you like to write a feedback?"* and provides *"Yes"* and *"No"* options. 4. If the user selects *"No"*, the system stops the notification from appearing. 5. If the user selects *"Yes"*, the system redirects them to the feedback page. 6. The user writes a comment and rates the internship experience. 7. The user clicks the *"Post"* button to submit the feedback. 8. The system processes and forwards the student's and company member's tags along with the internship id to the relevant external tool.
Exit Condition	The user either refuses to write a feedback or posts it
Exceptions	The feedback is incomplete, the system shows an error message.

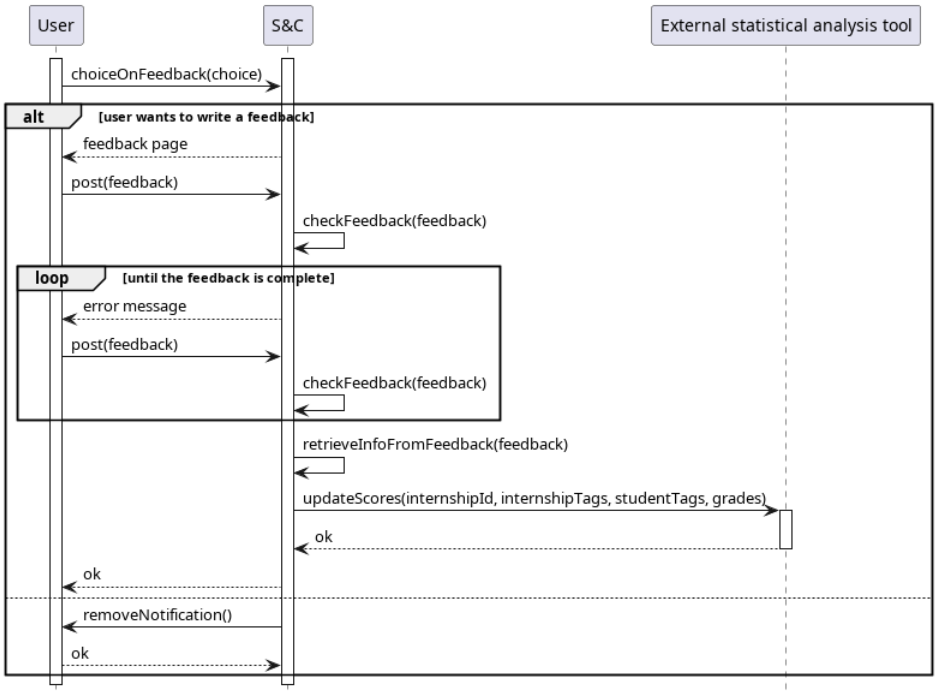

Figure 39: Sequence diagram of *"Student or company member posts a feedback"* use case.

[UC29] - The system closes the internship

Name	The system closes the internship
Actors	External statistical analysis tool
Entry Condition	The application period of an internship is over.
Event Flow	• The system closes the application period of an internship. • The system notifies the external statistical analysis tool about the closed application period.
Exit Condition	The internship announcement is no more visible and the external statistical analysis tool no more suggests the internship.
Exceptions	

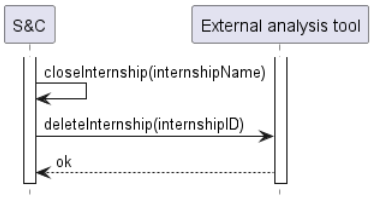

Figure 40: Sequence diagram of *"The system closes the internship"* use case.

3.4 Mapping on requirements

This section serves as a bridge between the high-level goals of the system and the specific functionalities required to achieve them. It also maps use cases with requirements and domain assumptions.

For each goal, the corresponding requirements are listed.

Requirement	**G1**	**G2**	**G3**	**G4**	**G5**	**G6**	**G7**
R1	✓	✓	✓	✓	✓	✓	✓
R2	✓	✓	✓	✓	✓	✓	✓
R3	✓	✓	✓	✓	✓	✓	✓
R4	✓	✓	✓	✓	✓	✓	✓
R5	✓	✓	✓	✓	✓	✓	✓
R6	✓	✓	✓	✓	✓	✓	✓
R7			✓	✓	✓	✓	✓
R8	✓	✓	✓	✓	✓	✓	✓
R9	✓	✓					
R10			✓				
R11		✓	✓				
R12	✓			✓			
R13				✓			
R14					✓	✓	
R15		✓	✓				
R16		✓	✓				
R17						✓	
R18					✓		✓
R19					✓		
R20	✓	✓					
R21					✓		✓
R22						✓	
R23				✓			
R24	✓			✓			
R25						✓	
R26					✓		✓
R27						✓	
R28						✓	
R29						✓	
R30						✓	
R31						✓	
R32						✓	
R33	✓						
R34						✓	
R35					✓		

For each use case, the corresponding requirements and domain assumptions are listed.

Use Case	Requirements	Domain assumptions
UC1	R3, R4	DA10, DA12
UC2	R2, R5	DA10, DA13
UC3	R1	DA10, DA11
UC4	R6	
UC5	R6	
UC6	R6	
UC7	R8	DA5
UC8	R24	DA5
UC9	R33	
UC10	R11, R20	DA1
UC11	R23	DA2
UC12	R7	DA2
UC13	R9, R10, R11	DA1
UC14	R13, R23	DA2
UC15	R12, R24	DA5
UC16	R27, R28	
UC17	R29, R30	
UC18	R32	
UC19	R32	
UC20	R17, R25	DA4, DA8, DA9
UC21	R14	DA7
UC22	R34	
UC23	R25	
UC24	R18, R26	
UC25	R18, R26	
UC26	R35	
UC27	R19	DA3
UC28	R15, R16	
UC29	R22	

3.5 Performance Requirements

3.5.1 Response time

The system's response time must be minimized to ensure an optimal and seamless user experience, even during peak load periods.

3.5.2 Scalability

The system must be scalable to ensure continued functionality in the future, following an increase in users. As an application that could potentially be used by the majority of students and companies, the system must be able to handle a growing number of users without compromising performance. Scalability must be ensured both horizontally and vertically.

3.5.3 Data volume

The system must support the permanent storage of user information for a large number of students from various universities and company/university members. It should ensure high performance and responsiveness, even as the volume of stored data grows significantly.

3.5.4 Latency

The latency of communication with external tools (e.g., video conferencing tools and statistical analysis tools) and with end-user devices must be low, ensuring a seamless user experience. The maximum acceptable latency for critical operations should be below 200 ms. An example of a critical operation is sending the communication of internship termination, due to the student's university decision.

3.6 Design Constraints

3.6.1 Privacy Complaints

All users' sensible information must be handled accordingly to existing regulations (e.g GPDR).

3.6.2 Hardware limitations

- Web Application
  - Internet connection
  - Device with a display
  - Web browser
- Backend
  - Scalable webserver
  - Database Management System

3.7 Software System Attributes

3.7.1 Reliability

The system must be fault and byzantine tolerant to prevent the system from propagating erroneous information to the user and along the internal components.

3.7.2 Availability

The system must be as available as possible, with a minimum value of 99.9% of time. It shall prevent maintenance break during the day, limiting such breaks only at nightime when it is unlike to have users interacting with the platform.

3.7.3 Security

The system must validate any user accessing the platform, always verifying the permissions of each user's operation. Measures to protect the database from malicious attacks must be adopted. The user's personal data must be encrypted.

3.7.4 Maintainability

All the code produced should be well commented and tested. The system structure should provide the ability to deploy updates in the backend without the user noticing it. System components should be created to be as modular as possible.

3.7.5 Portability

The system must be accessible by the Users from every kind of Web Browser.

4 Formal analysis using Alloy

This is an Alloy representation of the system described. The model does not include all the classes present in the class diagram but mainly focuses on the application phase, during which requests are handled.

In some cases, Alloy signatures have different attributes compared to their counterparts in the class diagram. These differences are necessary to provide a more comprehensible representation and do not affect the system's behavior.

A fundamental observation is that this Alloy model does not aim to represent a complete scenario. While, in a real system, users, internships, and other entities modeled through signatures could be added or modified, in this representation we assume a fixed set for each of these entities. Based on these static sets, we focus on modeling the behavior of requests, which is the only dynamic signature in the model.

```
-------------------------------------------------------------------------
-- SIGNATURES
-------------------------------------------------------------------------

-- USER


abstract sig User{
        userInformation : UserPersonalInformation,
        credentials : UserLoginCredentials
}


-- Contains the user's email and password.

sig UserLoginCredentials{ }


sig UserPersonalInformation{ }


sig CompanyMember extends User{
        employedAtCompany: Company
}


sig UniversityMember extends User{
        employedAtUniversity: University
}


sig Student extends User{
        enrolled: University
}


-- GROUP OF USERS


-- In order to be created, a University must have at least one member.

sig University{
        universityInformation: UniversityInformation,
        universityMailDomain: MailDomain,
}{ some member : UniversityMember | this in member.employedAtUniversity }


-- Contains university information, such as its name.
```

```
sig UniversityInformation{
}


-- In order to be created, a Company must have at least one member.

sig Company{
        companyInformation: CompanyInformation,
        companyMailDomain: MailDomain,
}{ some member : CompanyMember | this in member.employedAtCompany }


-- Contains company information, such as its name and a description.

sig CompanyInformation{ }


-- Represents the email domain of a company or university and identifies them.

sig MailDomain{ }


-- INTERNSHIP


sig Internship{
        id : InternshipId,
        offered: Company,
        description: InternshipDescription,
        var candidatureStatus: CandidaturesStatus
}


-- Represents an ID used to distinguish between different internship offers.

sig InternshipId { }


-- Defines the interval during which applications for a specific internship can be
    ↪  submitted. According to the class diagram attributes,
-- the internship will have a status of 'Open' between the applicationStartDate
    ↪ and applicationEndDate, and 'Closed' at all other times.

enum CandidaturesStatus{ Closed, Open }


-- Contains all information related to the internship offer, including the name,
    ↪ terms, and duration.

sig InternshipDescription{ }


--REQUEST


-- This signature is dynamic over time, as different requests can be sent at
    ↪ different moments.

var sig Request{
        var requestStatus: RequestStatus,
        var internshipReferred: Internship,
        var requestedBy: User,
        var requestedTo: User
}
```

```
enum RequestStatus{ Pending, Approved, Declined }


-------------------------------------------------------------------------
-- PREDICATES
-------------------------------------------------------------------------


-- The predicate is true when the internship is open for receiving applications.

pred internshipOpenForApplications[ i : Internship ]{
        i.candidatureStatus = Open
}


-- The predicate models the closure of an internship period for receiving
    ↪ applications.

pred closeApplicationAvailability[ i : Internship ]{
        i.candidatureStatus = Open and i.candidatureStatus' = Closed
}


-- The predicate models the opening of an internship period for receiving
    ↪ applications (the internship may also be published as already open).

pred openApplicationAvailability[ i : Internship ]{
        i.candidatureStatus = Closed and i.candidatureStatus' = Open
}


-- The predicate models the rejection of a pending request.

pred declineRequest[ r : Request ]{
        r.requestStatus = Pending
        r.requestStatus' = Declined
}


-- The predicate models the approval of a pending request.

pred approveRequest[ r : Request ]{
        r.requestStatus = Pending
        r.requestStatus' = Approved
}


-- The predicate represents the addition of a new request to the model.

pred addNewRequest [ sender , receiver : User , internship : Internship ]{

        //precondition

        internship in Internship
        sender in (Student + CompanyMember)
        receiver in (Student + CompanyMember)
        ( ( sender in Student ) implies ( receiver in CompanyMember ) )
        ( ( sender in CompanyMember ) implies ( receiver in CompanyMember ) )

        //postcondition

        ( no r : Request |
                r.internshipReferred = internship and
                r.requestedBy = sender and
                r.requestedTo = receiver )
        ( after (
                one r : Request |
                        r.internshipReferred = internship and
                        r.requestedBy = sender and
```

```
                        r.requestedTo = receiver and
                        r.requestStatus = Pending ) //say redundant
        )

}



-------------------------------------------------------------------------
-- FACTS
-------------------------------------------------------------------------



-- For all internships, once the period for accepting applications ends, it cannot
    ↪  be reopened.

fact applicationAvailabilityAfterClosure{
        all i : Internship |
                always (
                        ( i.candidatureStatus = Closed and
                          once closeApplicationAvailability[ i ] )
                                implies i.candidatureStatus' = Closed
                )
}



-- Every internship that has never been available for receiving applications will
    ↪ eventually enter a period of availability.

fact applicationAvailabilityBeforeOpening{
        all i : Internship |
                always (
                        ( historically  i.candidatureStatus = Closed )
                                implies ( eventually  i.candidatureStatus = Open )
                )
}



-- Every internship's period of availability for receiving applications will
    ↪ eventually end.

fact applicationAvailabilityAfterOpening{
        all i : Internship |
                always (
                        ( i.candidatureStatus = Open )
                                implies ( eventually  i.candidatureStatus = Closed
                                    ↪  )
                )
}



-- There are not two instances of the same internship offer

fact credentialsIdentifyUsers{
        no disj u1, u2 : User |
                 u1.credentials = u2.credentials
}



-- There are not two instances of the same Company

fact mailDomainIdentifiesCompanies{
        no disj c1, c2 : Company |
                c1.companyMailDomain = c2.companyMailDomain
}



-- There are not two instances of the same University

fact mailDomainIdentifiesUniversities{
```

```
        no disj u1, u2 : University |
                u1.universityMailDomain = u2.universityMailDomain
}


-- There are not common email domain between universities and companies

fact noCompanySharesMailDomainWithUniversity{
        no c : Company , u : University |
                c.companyMailDomain = u.universityMailDomain
}


-- Every pending request will eventually be evaluated.

fact pendingRequestBehaviour{
        always (
                all r : Request |
                        r.requestStatus = Pending
                                implies ( ( ( eventually  declineRequest[ r ] ) or
                                        ( eventually  approveRequest[ r ] ) ) )
        )
}


-- An evaluated request cannot be evaluated again or return pending.

fact requestAreEvaluatedOnlyOneTime{
        always (
                all r : Request |
                        r.requestStatus ≠ Pending
                                implies r.requestStatus' = r.requestStatus
        )
}


-- All the evaluated request were once pending. //redundant

fact evaluatedRequestWerePending{
        always (
                all r : Request |
                        r.requestStatus ≠ Pending
                                implies once r.requestStatus = Pending)
}


-- There cannot be a Request which sender is also the receiver.

fact usersCannotSendRequestToThemselves{
        always (
                all r : Request |
                        r.requestedBy ≠ r.requestedTo
        )
}


-- There are not duplicate requests.

fact noDuplicateRequests{
        always (
                all disj r1, r2 : Request |
                        (r1.requestedBy ≠ r2.requestedBy) or
                        (r1.internshipReferred ≠ r2.internshipReferred) )
}


-- The sender and receiver must always be either a Student and a CompanyMember, or
    ↪  a CompanyMember and a Student.

fact userRolesConstraints{
        always (
                 all r : Request |
```

```
                        # ( ( r.requestedBy + r.requestedTo ) & Student ) = 1 and
                        # ( ( r.requestedBy + r.requestedTo ) & CompanyMember ) =
                            ↪ 1
        )
}



-- There are not duplicate internship offers

fact noDuplicateInternship{
        no disj i1, i2 : Internship | i1.id = i2.id
}



-- The Request signature is dynamic, but the fields "internshipReferred", "
    ↪ requestedBy", and "requestedTo" of a specific request
-- do not change over time once assigned.

fact requestFieldsDoesNotChange{
        always (
                all r : Request |
                        r.internshipReferred' = r.internshipReferred and
                        r.requestedBy' =  r.requestedBy and
                        r.requestedTo' = r.requestedTo
        )
}



-- All requests sent by students are evaluated after the application period
    ↪ terminates, once the company has received all possible applications.
-- If a company member sent the request to a student, the student has no
    ↪ constraint on when to evaluate the request.

fact requestEvaluatedAfterClosure{
        always(
                all r : Request |
                        ( r.requestStatus ≠ Pending and
                          r.requestedBy in Student )
                                implies r.internshipReferred.candidatureStatus =
                                    ↪ Closed
        )
}



-- It is not possible to have a request related to an internship that has not
    ↪ started its period of availability for receiving applications.
-- Requests begin to arrive only after the period starts.

fact requestCannotReferInternshipsBeforeOpening{
        always(
                all r : Request |
                        r.internshipReferred.candidatureStatus = Closed
                                implies ( once closeApplicationAvailability[ r.
                                    ↪ internshipReferred ] )
        )
}



-- Every Request has been added to the model once

fact allRequestHaveBeenOnceSent{
        always (
                all r : Request |
                        ( once ( not r in Request ) ) and
                        ( once ( addNewRequest [ r.requestedBy, r.requestedTo, r.
                            ↪ internshipReferred ] ) )
                )

}
```

```
-------------------------------------------------------------------------
-- FUNCTIONS
-------------------------------------------------------------------------


-- Returns all the company members of a Company "c".

fun companyMembersEmployed[ c : Company ] : some CompanyMember{
        { m : CompanyMember | m.employedAtCompany = c }
}


-- Returns all the university members of a University "u",

fun universityMembersEmployed[ u : University ] : some UniversityMember{
        { m : UniversityMember | m.employedAtUniversity = u }
}


-- Returns all the students of a University "u".

fun studentsEnrolled[ u : University ] : set Student{
        { s : Student | s.enrolled = u }
}


-------------------------------------------------------------------------
-- RUN EXAMPLES
-------------------------------------------------------------------------


pred show1 {
        #Request' = 2
        #Internship = 2
        #Company = 2
        #University = 1
        #Student = 2
        #CompanyMember = 2
}

pred show2 {
        ( some c1, c2 : User, i : Internship | eventually addNewRequest[ c1, c2, i
            ↪  ] )
        eventually ( some r1, r2 : Request | r1.requestStatus = Approved and r2.
            ↪ requestStatus = Declined )
        #Internship > 2
        #Company = 2
        #University = 2
        ( some i : Internship | i.candidatureStatus = Closed )
}

run show1 for 10
```

4.1 Pred Show1 Run Example

At this initial stage, the two internships are instantiated, but no requests have been added yet.

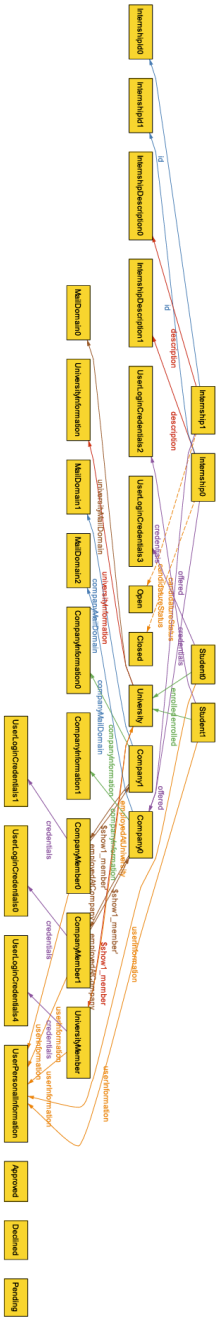

Figure 41: show1 first iteration

At the second instant, two requests are sent, both referring to Internship0, which was closed in the previous instant but is now open. The two requests are both in a Pending state: one is sent by a company member to a student, and the other by a student to a company member.

Figure 42: show1 two iteration

At the third instant, both requests are evaluated and declined.

Figure 43: show1 three iteration

5 Effort spent

Section	Matteo Gatti	Alessio Ginolfi	Riccardo Cerberi
Section 1	12	15	13
Section 2	16	10	17
Section 3	18	8	15
Section 4	2	16	2

6 References

Below there is a list of all the tools, libraries and sources used to compose this document.

- **PlantUML**: a tool used to create UML diagrams (class diagrams, sequence diagrams, use case diagrams and state diagrams) by writing textual descriptions.
- ***alloy-style.sty* LaTeX library**: an open LaTeX library that provides a set of styles for formatting Alloy model code within the document. This library has been modified to add support for the temporal keywords of Alloy 6, such as *always* and *after*.
- ***tikz-uml.sty* LaTeX library**: an open LaTeX library for creating UML diagrams directly in LaTeX documents using the TikZ package.
- **MiKTeX**: A distribution of LaTeX for Windows, used to compile and render the LaTeX code into a PDF document.
- **Visual Studio Code**: A source-code editor used for writing, editing, and managing the LaTeX document and related code.
- **LaTeX Workshop extension for VS Code**: An extension for Visual Studio Code that adds support for LaTeX documents.
- **Strawberry PERL**: A Perl distribution required by LaTeX tools.
- **Alloy Analyzer 6.1.0**: a tool for defining, visualizing, and analyzing the formal specification of the system.
- **GitHub**: a platform used for version control and collaborative work, where the LaTeX files and other resources were stored and managed.